

TRƯỜNG ĐẠI HỌC THỦY LỢI
KHOA CÔNG NGHỆ THÔNG TIN



GIÁO TRÌNH

THỰC HÀNH PHÁT TRIỂN ỨNG DỤNG CHO THIẾT BỊ DI ĐỘNG

Hà Nội, 2.2025

MỤC LỤC

CHƯƠNG 1. LÀM QUEN	4
Bài 1) Tạo ứng dụng đầu tiên	4
1.1) Android Studio và Hello World	4
1.2) Giao diện người dùng tương tác đầu tiên	26
1.3) Trình chỉnh sửa bố cục	26
1.4) Văn bản và các chế độ cuộn	41
1.5) Tài nguyên có sẵn	41
Bài 2) Activities	41
2.1) Activity và Intent	41
2.2) Vòng đời của Activity và trạng thái	41
2.3) Intent ngầm định	41
Bài 3) Kiểm thử, gỡ lỗi và sử dụng thư viện hỗ trợ	41
3.1) Trình gỡ lỗi	41
3.2) Kiểm thử đơn vị	41
3.3) Thư viện hỗ trợ	41
CHƯƠNG 2. TRẢI NGHIỆM NGƯỜI DÙNG	42
Bài 1) Tương tác người dùng	42
1.1) Hình ảnh có thể chọn	42
1.2) Các điều khiển nhập liệu	42
1.3) Menu và bộ chọn	42
1.4) Điều hướng người dùng	42
1.5) RecyclerView	42
Bài 2) Trải nghiệm người dùng thú vị	42
2.1) Hình vẽ, định kiểu và chủ đề	42
2.2) Thẻ và màu sắc	42

2.3)	Bố cục thích ứng	42
Bài 3)	Kiểm thử giao diện người dùng.....	42
3.1)	Espresso cho việc kiểm tra UI	42
CHƯƠNG 3. LÀM VIỆC TRONG NỀN		42
Bài 1)	Các tác vụ nền	42
1.1)	AsyncTask.....	42
1.2)	AsyncTask và AsyncTaskLoader	42
1.3)	Broadcast receivers.....	42
Bài 2)	Kích hoạt, lập lịch và tối ưu hóa nhiệm vụ nền	42
2.1)	Thông báo	42
2.2)	Trình quản lý cảnh báo	42
2.3)	JobScheduler	42
CHƯƠNG 4. LƯU DỮ LIỆU NGƯỜI DÙNG		43
Bài 1)	Tùy chọn và cài đặt	43
1.1)	Shared preferences	43
1.2)	Cài đặt ứng dụng	43
Bài 2)	Lưu trữ dữ liệu với Room	43
2.1)	Room, LiveData và ViewModel	43
2.2)	Room, LiveData và ViewModel	43
3.1)	Trình gowx loi	

CHƯƠNG 1. LÀM QUEN

Bài 1) Tạo ứng dụng đầu tiên

1.1) Android Studio và Hello World

Giới thiệu

Trong bài thực hành này, bạn sẽ tìm hiểu cách cài đặt Android Studio, môi trường phát triển Android. Bạn sẽ tạo và chạy ứng dụng Android đầu tiên của mình, Hello World, bằng giả lập và điện thoại thật.

Những gì Bạn nên biết

Bạn nên có khả năng:

- Hiểu quy trình phát triển phần mềm chung cho các ứng dụng lập trình hướng đối tượng sử dụng một IDE (môi trường phát triển tích hợp) như Android Studio.
- Chứng minh rằng bạn có ít nhất 1-3 năm kinh nghiệm trong lập trình hướng đối tượng và có kinh nghiệm nhất định với Java. (Các bài thực hành này sẽ không giải thích về lập trình hướng đối tượng hoặc Java.)

Những gì bạn cần chuẩn bị:

- Máy tính chạy Windows hoặc Linux hoặc Mac. Vào trang tải Android Studio để xem cấu hình máy tính yêu cầu mới nhất.
- Vào internet hoặc cách nào đó tải Android Studio và Java bản mới nhất về máy.

Những gì bạn sẽ học được:

- Cách cài đặt và sử dụng IDE Android Studio.
- Quy trình phát triển để xây dựng ứng dụng Android.
- Cách tạo một dự án Android từ một mẫu.
- Cách logging để gỡ lỗi.

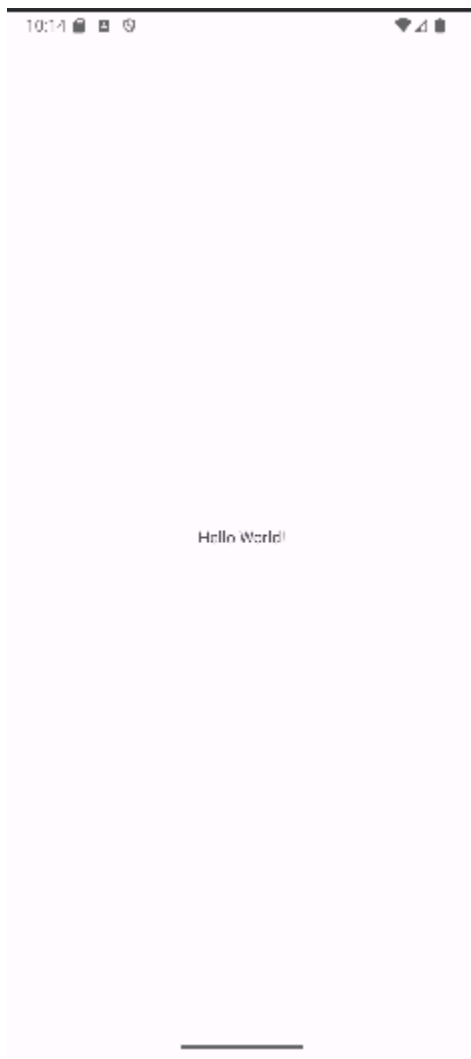
Những gì bạn sẽ làm

- Cài đặt môi trường phát triển **Android Studio**.
- Tạo một trình giả lập (thiết bị ảo) để chạy ứng dụng của bạn trên máy tính.
- Tạo và chạy ứng dụng **Hello World** trên thiết bị ảo và vật lý.
- Khám phá cấu trúc dự án.

- Tạo và xem log từ ứng dụng của bạn.
- Khám phá file **AndroidManifest.xml**

Tổng quan về ứng dụng:

Sau khi bạn cài đặt thành công Android Studio, bạn sẽ tạo một dự án mới cho ứng dụng Hello World từ một template (mẫu). Ứng dụng đơn giản này hiển thị dòng chữ "Hello World" trên màn hình của thiết bị Android ảo hoặc thiết bị vật lý.



Task 1: Tải android studio

Android Studio cung cấp một môi trường phát triển tích hợp (IDE) hoàn chỉnh, gồm một trình soạn thảo code nâng cao và một tập hợp các ứng dụng mẫu. Nó còn chứa các công cụ để phát triển, gỡ lỗi, kiểm thử và tối ưu hiệu năng, giúp phát triển ứng dụng nhanh chóng và dễ dàng hơn. Bạn có thể test ứng dụng với loạt các trình giả lập được cấu hình sẵn hoặc test trên điện thoại của bạn, tạo ứng dụng và đẩy lên Google Play Store.

Lưu ý: Android Studio liên tục được cải thiện. Để biết thông tin mới nhất về các yêu cầu hệ thống và hướng dẫn cài đặt, tham khảo [Android Studio](#).

Android Studio đã hỗ trợ Windows, Linux hoặc Mac. Android Studio tích hợp sẵn **OpenJDK** (Bộ công cụ phát triển Java) mới nhất.

Để sử dụng Android Studio, trước tiên kiểm tra [yêu cầu hệ thống](#) để đảm bảo rằng hệ thống của bạn đáp ứng sử dụng Android Studio. Cài đặt tương tự các nền tảng khác. Lưu ý những điểm khác biệt dưới đây.

1. Truy cập [trang web dành cho android developer](#) và làm theo hướng dẫn để tải xuống và cài đặt Android Studio.
2. Chấp nhận các cấu hình mặc định ở các bước và đảm bảo chọn các thành phần để cài đặt.
3. Sau khi cài đặt xong, trình thiết lập (Setup Wizard) sẽ tải xuống và cài đặt một số thành phần bổ sung bao gồm Android SDK. Kiên nhẫn, quá trình này có thể mất một lúc tùy tốc độ Internet của bạn và một số bước có vẻ thừa.
4. Khi cài đặt hoàn tất, Android Studio sẽ khởi động và bạn có thể tạo dự án đầu tiên của mình.

Khắc phục sự cố: Nếu gặp vấn đề trong khi cài đặt, hãy kiểm tra các ghi chú phát hành của Android Studio hoặc tìm người giúp.

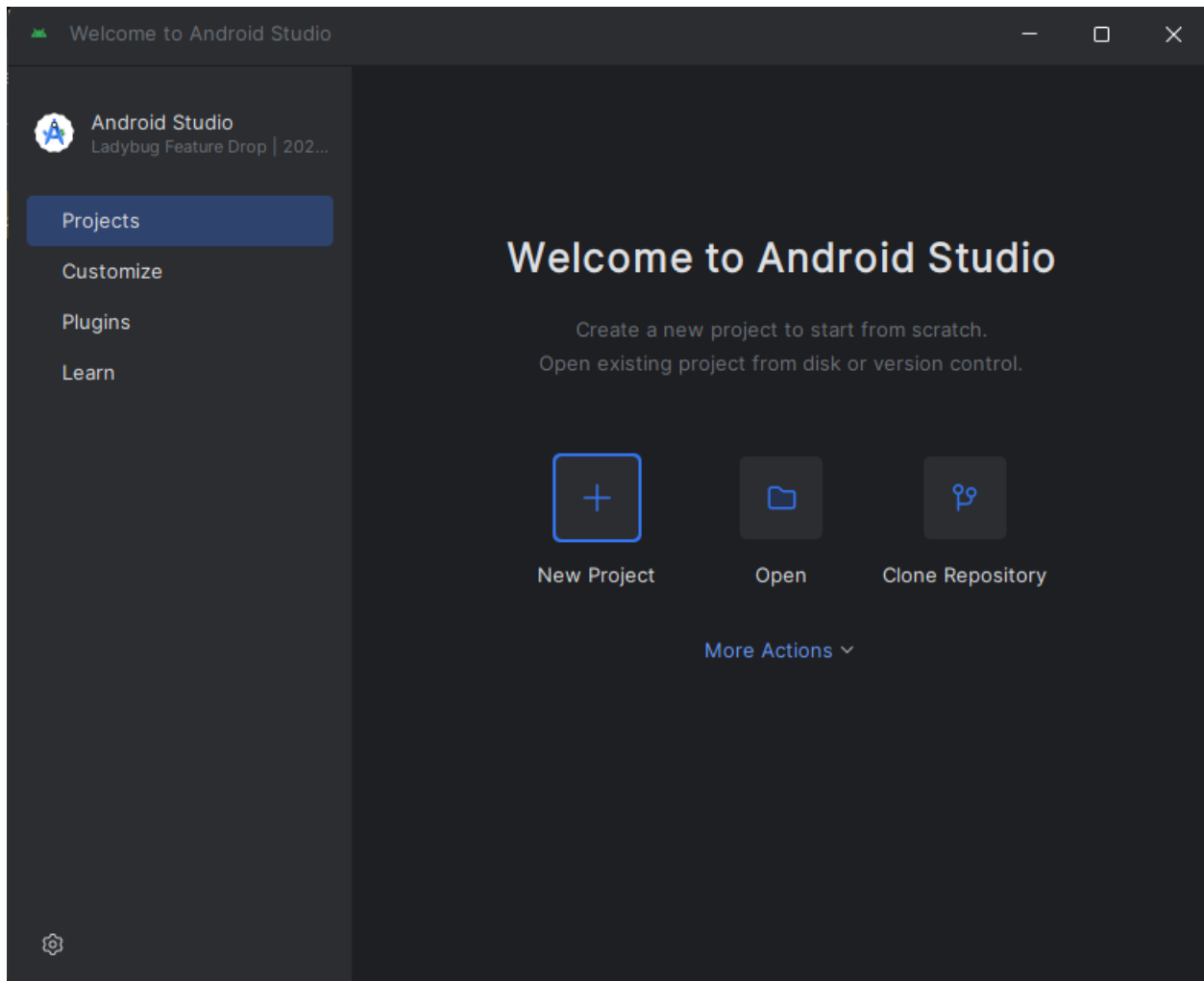
Task 2: Tạo ứng dụng Hello World

Trong nhiệm vụ này, bạn sẽ tạo một ứng dụng hiển thị "Hello World" để xác nhận rằng Android Studio đã được cài đặt thành công và để học phát triển ứng dụng cơ bản với Android Studio.

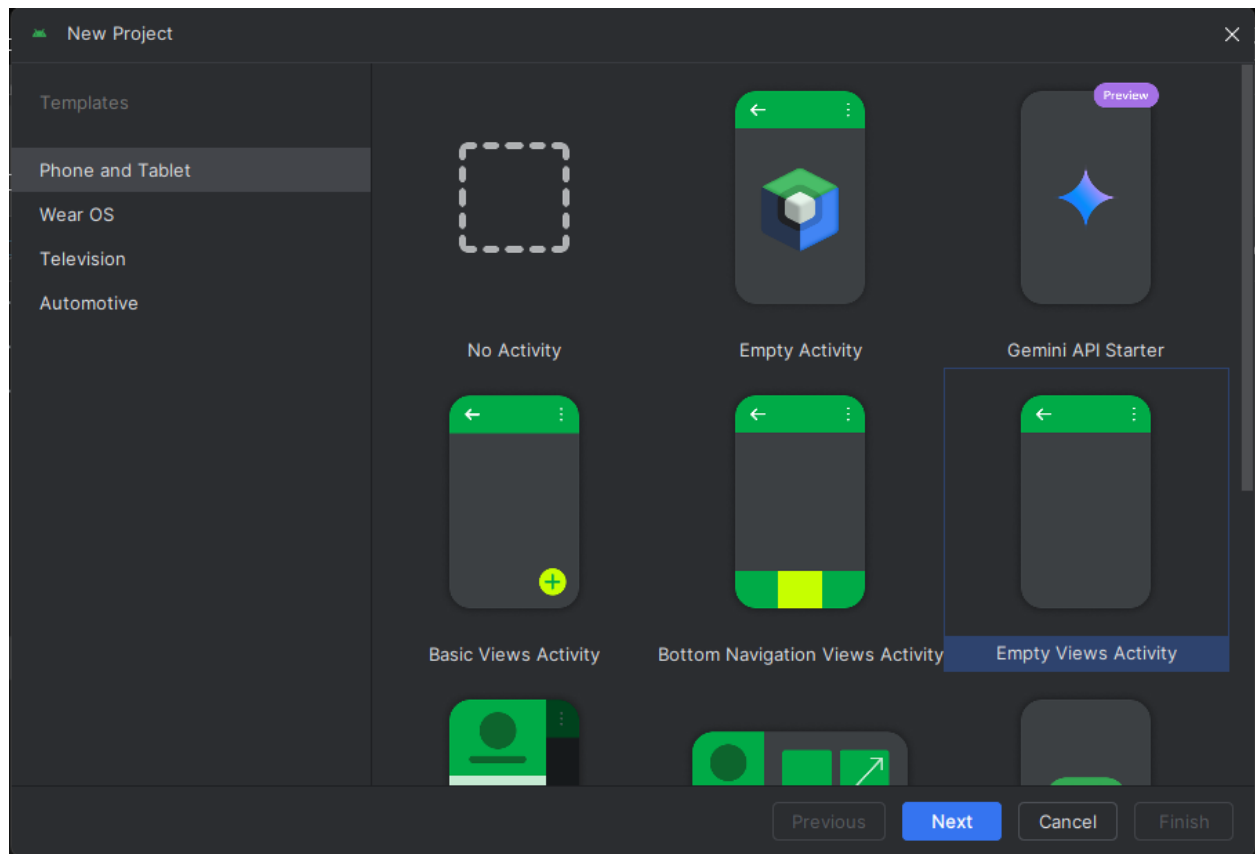
2.1 Tạo dự án

1. Mở Android Studio nếu chưa mở

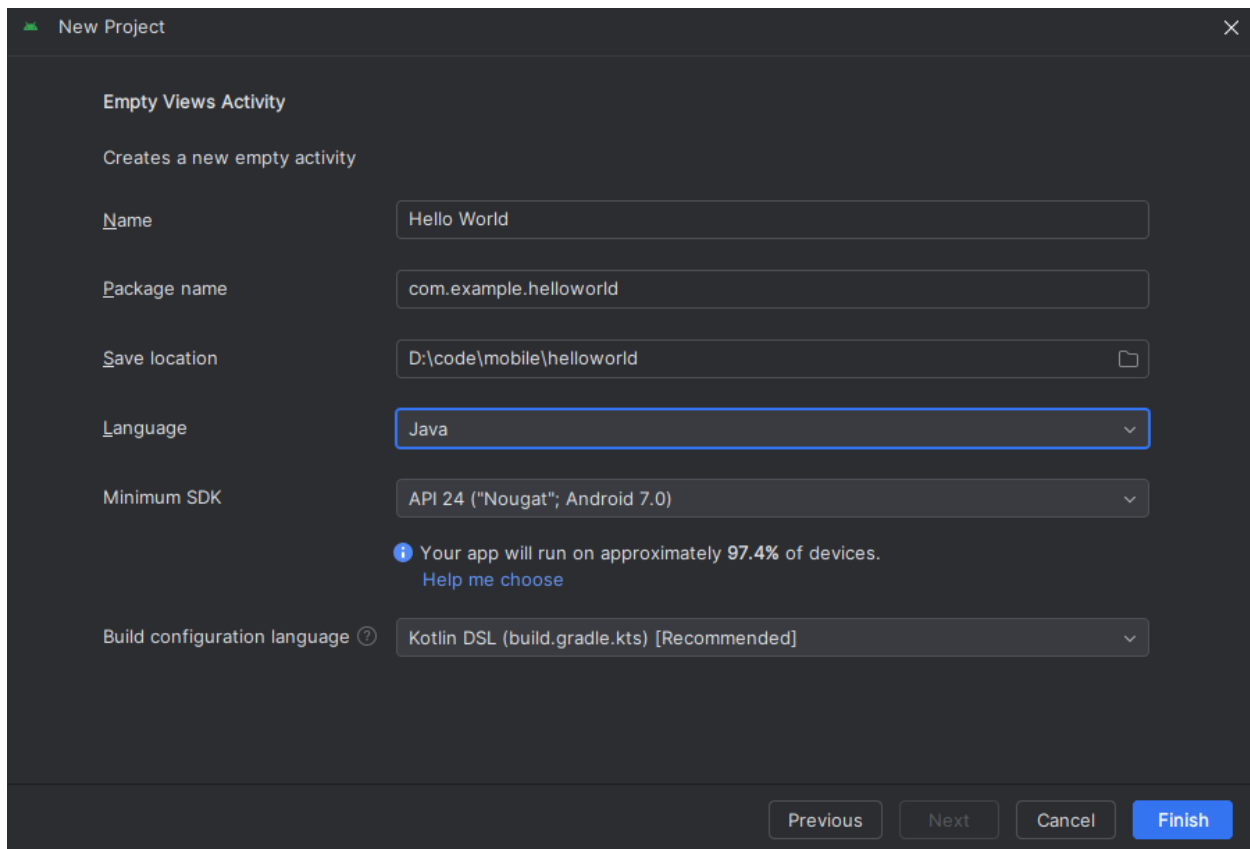
2. Ở cửa sổ **Welcome to Android Studio**, trong **Projects** ấn vào **New Project**.



3. Chọn 1 template phù hợp với dự án của bạn rồi **Next**. Ở đây chúng ta chọn **Empty View Activity** để bắt đầu.



4. Tiếp theo nhập **Hello Word** vào trường **Name** để đặt tên cho ứng dụng, **Language** là **Java**, **Minium SDK** nên sử dụng recommend sẽ tiếp cận được với nhiều người sử dụng nhất. Package name thường đặt với tên miền công ty nên ở đây để mặc định.



5. Cuối cùng ấn **Finish**. Và chờ setup môi trường cho dự án.

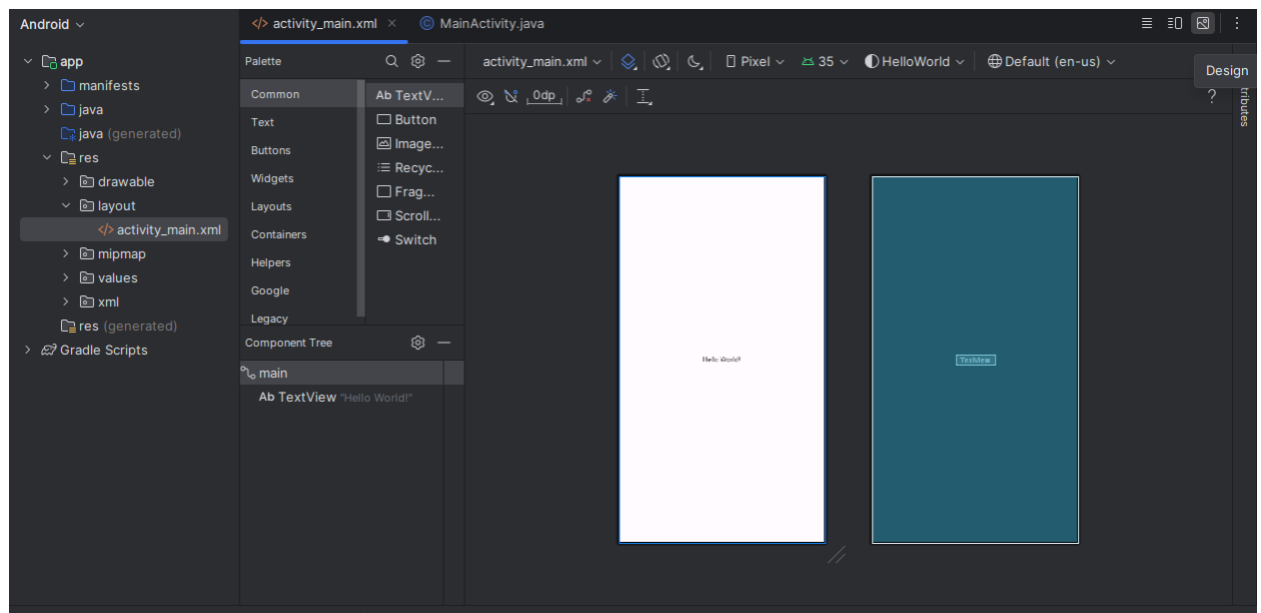
Android Studio tạo một thư mục cho các dự án của bạn và xây dựng dự án bằng Gradle (quá trình này có thể mất vài phút).

Mẹo: Xem [trang nhà phát triển](#) để biết thông tin chi tiết.

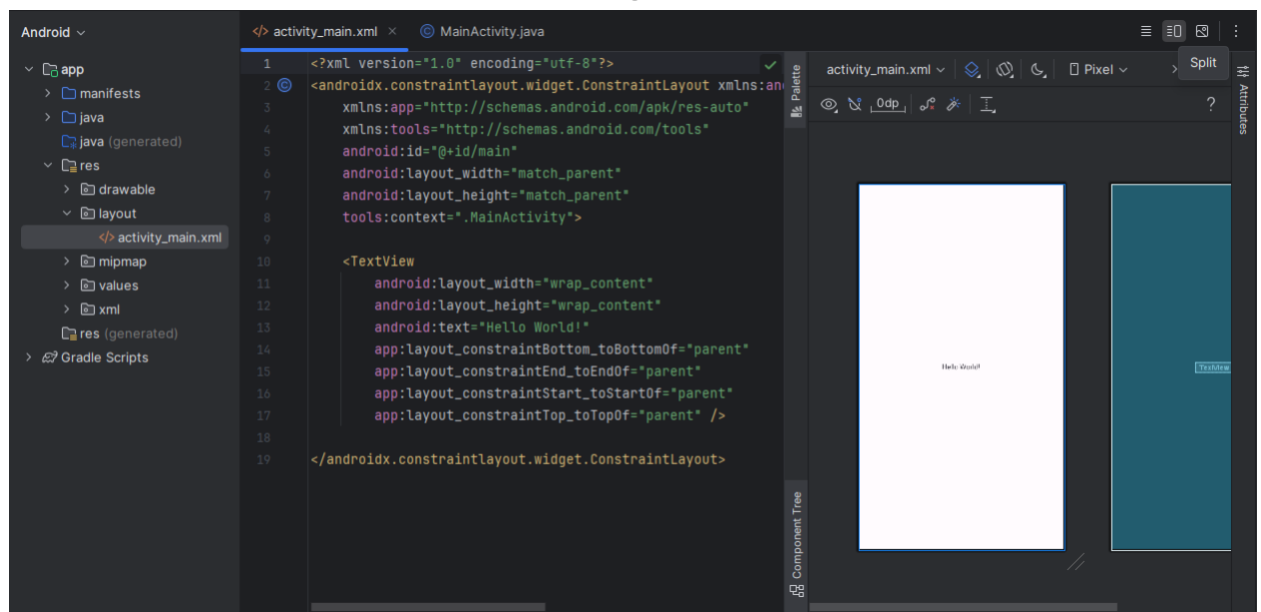
Bạn cũng có thể thấy thông báo "Tip of the day" và các phím tắt và các mẹo hữu ích khác. Nhấp vào **Close** để đóng thông báo.

Trình chỉnh sửa của Android Studio:

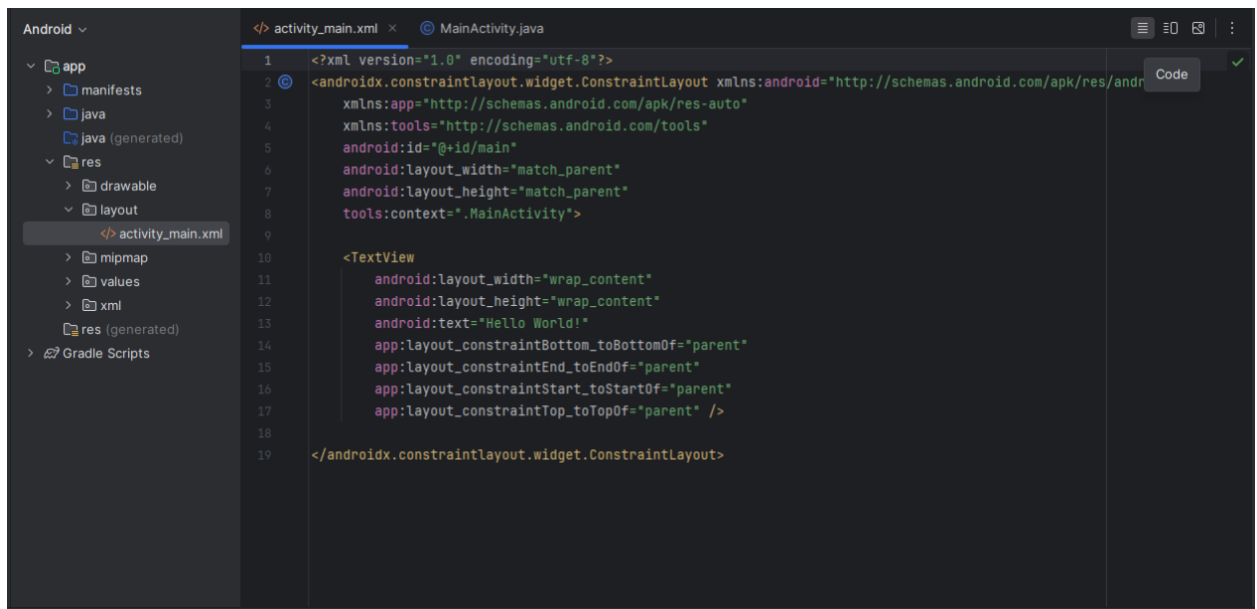
1. Vào **res/layout/activity_main.xml** để mở trình sửa layout, các file xml để code xử lý UI
 - 1.1 Chế độ trỏ chuột trở là chế độ **Design** để thao tác với các component bằng giao diện



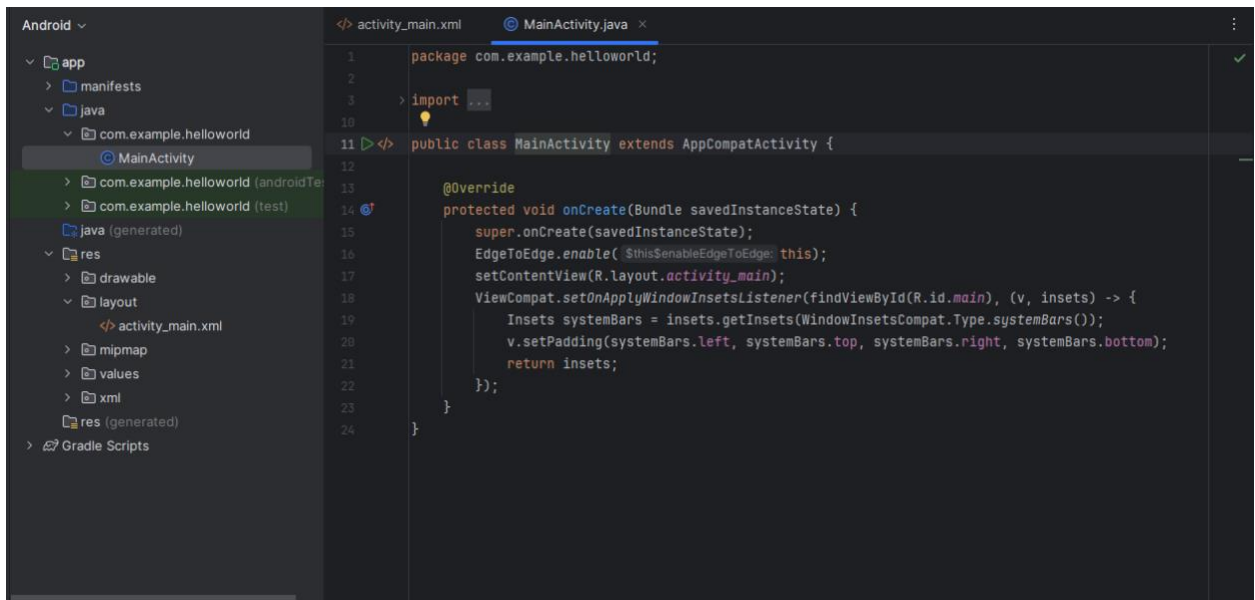
1.2 Chế độ **Split** với nửa màn hình chỉnh sửa giao diện và nửa code



1.3 Chế độ **Code** full màn hình code



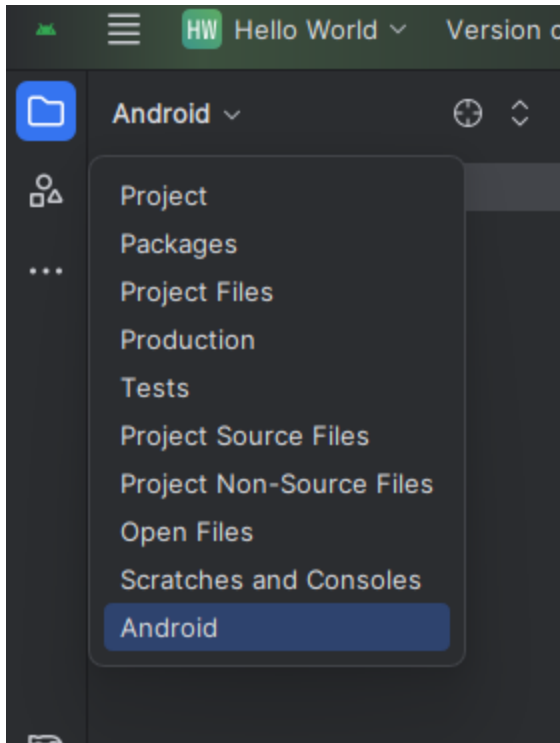
2. Vào `java/com/example/helloworld/MainActivity.java` để truy cập code editor để code xử lý các event hay action.



2.2 Vọc project > Android Pane

Trong phần thực hành này, bạn sẽ khám phá cách tổ chức dự án trong Android Studio.

1. Nếu chưa được chọn, nhấp vào tab **Project** trong cột tab dọc ở bên trái phía trên cửa sổ Android Studio. **Project pane** sẽ xuất hiện.
2. Để xem dự án theo cấu trúc chuẩn của dự án Android, chọn **Android** từ menu pop-up của **Project pane**.

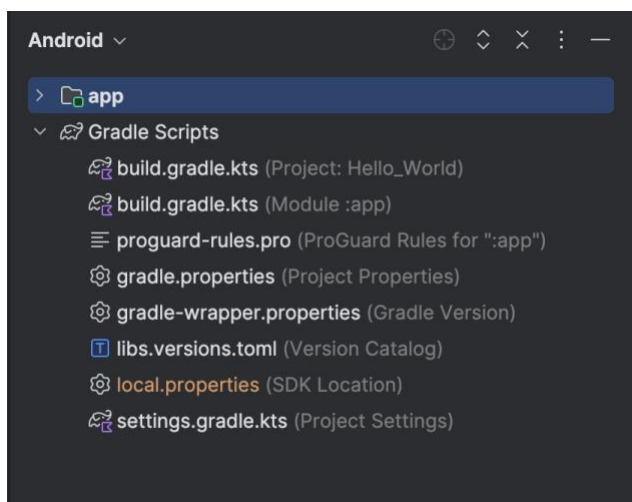


Lưu ý: Chương này và các chương khác đề cập đến **Project pane** với thiết lập **Project > Android pane**.

2.3 Khám phá thư mục Gradle Scripts

Gradle trong android studio giúp dễ dàng thêm các thư viện bên ngoài hoặc các module thư viện khác vào dự án như các dependency.

Khi vừa tạo dự án, cửa sổ **Project > Android** hiện ra với thư mục Gradle Scripts được mở sẵn.



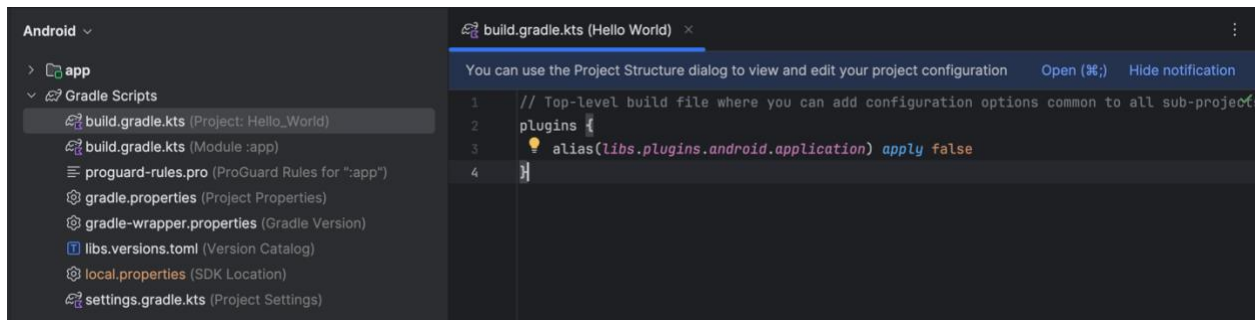
Làm theo các bước để làm quen với hệ thống Gradle:

1. Nếu thư mục Gradle Scripts chưa được mở, click vào hình tam giác để mở. Thư mục này chứa tất cả các file mà hệ thống build cần.

2. Tìm file **build.gradle.kts(Project: HelloWorld)**

Đây là nơi bạn sẽ thấy các tùy chọn cấu hình chung cho các module trong project. Mỗi project Android Studio đều có một file build Gradle cấp cao nhất (top-level). Thường thì không cần phải chỉnh sửa nhiều trong file này, nhưng hiểu được vẫn có ích hơn.

Mặc định, file build cấp cao nhất sẽ sử dụng khối lệnh plugins để khai báo các plugin Gradle cần thiết cho toàn bộ project. Các plugin chịu trách nhiệm build và đóng gói ứng dụng. Sử dụng cú pháp alias để tham chiếu đến các plugin đã được định nghĩa. Ví dụ như trong ảnh, `alias(libs.plugins.android.application)` `apply false` khai báo plugin android.application tổng quát, các plugin sẽ được sử dụng trong các file build.gradle riêng để dễ quản lý.



3. Tìm file **build.gradle.kts (Module:app)**

Ngoài file build.gradle.kts cấp project, mỗi module cũng có một file build.gradle.kts riêng. File này cho phép bạn cấu hình các thiết lập build cho từng module cụ thể (ứng dụng HelloWorld của chúng ta chỉ có một module). Việc cấu hình các thiết lập build này cho phép bạn tùy chỉnh các tùy chọn đóng gói, ví dụ như thêm các build type và product flavors. Bạn cũng có thể ghi đè các thiết lập trong file AndroidManifest.xml hoặc file build.gradle.kts cấp project.

File này thường được dùng để chỉnh sửa các cấu hình cấp ứng dụng (app-level), ví dụ như khai báo các dependencies trong phần dependencies. Bạn có thể khai báo dependency thư viện bằng một trong số các cấu hình dependency khác nhau. Mỗi

cấu hình dependency cung cấp cho Gradle các hướng dẫn khác nhau về cách sử dụng thư viện.

Ví dụ về file build.gradle.kts (Module:app) cho ứng dụng HelloWorld:

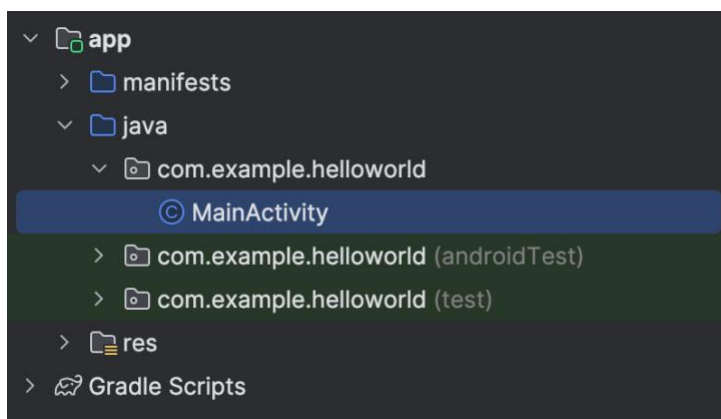
```
plugins {  
    alias(libs.plugins.android.application)  
}  
  
android {  
    ...  
  
    dependencies {  
  
        implementation(libs.appcompat)  
        implementation(libs.material)  
        implementation(libs.activity)  
        implementation(libs.constraintlayout)  
        testImplementation(libs.junit)  
        androidTestImplementation(libs.ext.junit)  
        androidTestImplementation(libs.espresso.core)  
    }  
}
```

4. Click vào hình tam giác để đóng Gradle Scripts lại.

2.4 Khám phá các thư mục app và res

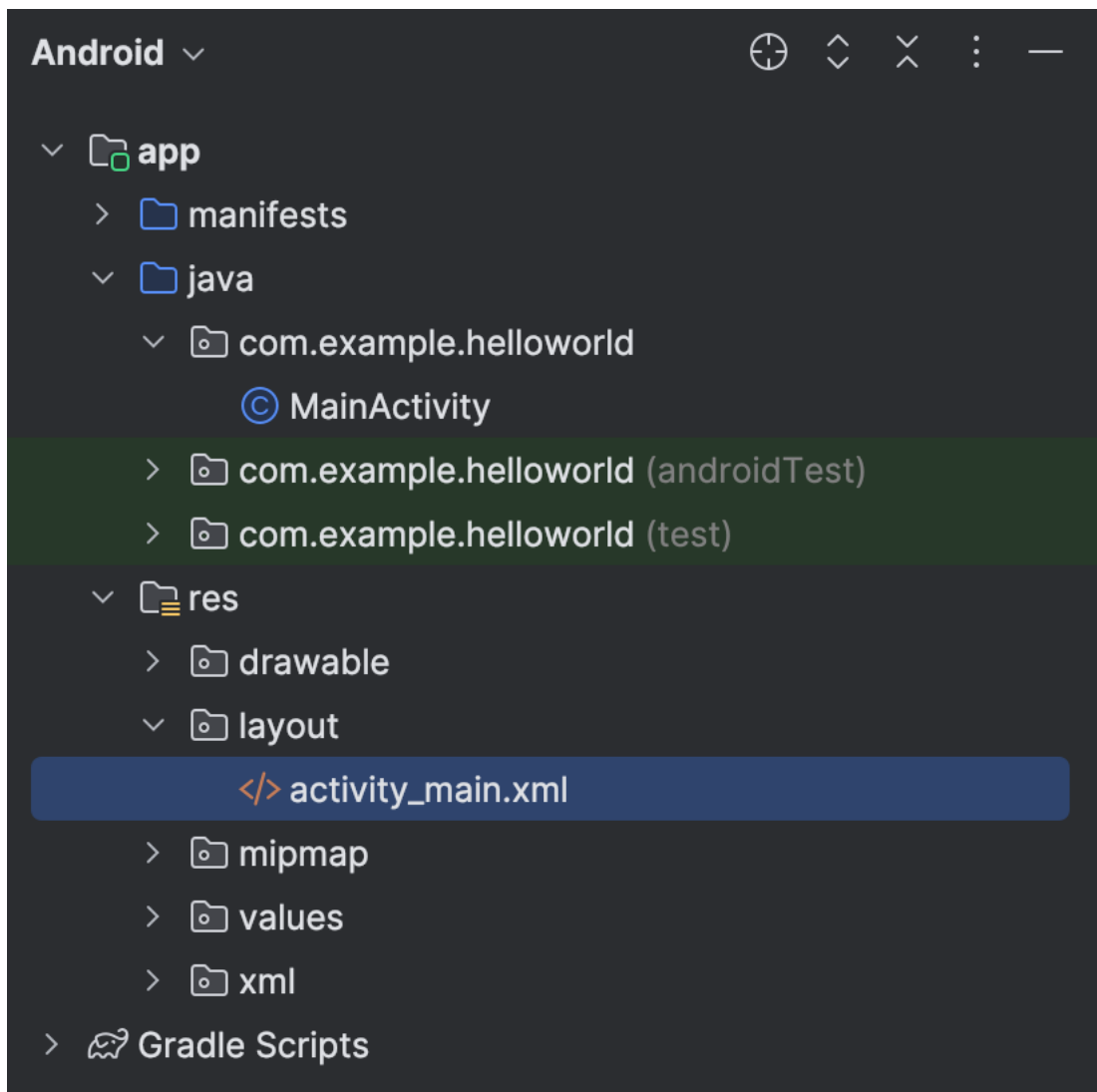
Tất cả code và tài nguyên cho ứng dụng đều nằm trong thư mục app và res.

1. Mở thư mục app, thư mục java, và thư mục com.example.helloworld để xem file Java MainActivity. Nhấn double vào MainActivity để mở nó trong trình soạn thảo code.



Thư mục java chứa các file class Java, được chia thành ba thư mục con như hình. Thư mục `com.example.hello.helloworld` (hoặc tên miền bạn đã chỉ định) chứa tất cả các file cho một package ứng dụng. Hai thư mục còn lại dùng để test và sẽ được mô tả trong bài học khác. Với ứng dụng Hello World, chỉ có một package và nó chứa file `MainActivity.java`. Tên của Activity đầu tiên (trong ảnh), cũng là nơi khởi tạo các tài nguyên của ứng dụng, thường được đặt tên là `MainActivity` (phần mở rộng file bị ẩn trong cửa sổ `Project > Android`).

2. Mở thư mục `res > layout`, double click vào `activity_main.xml` để mở trong trình soạn thảo layout.



Thư mục `res` chứa các tài nguyên, ví dụ như layout, chuỗi và hình ảnh. Một Activity thường liên kết với một layout của các view UI được định nghĩa trong một file XML. File này thường được đặt tên theo Activity của nó.

2.5 Khám phá thư mục manifests

Thư mục manifests chứa các file cung cấp thông tin quan trọng về ứng dụng cho hệ thống Android. Hệ thống phải có các thông tin này để chạy bất kỳ đoạn code nào của ứng dụng.

1. Mở rộng thư mục manifests.

2. Mở file AndroidManifest.xml.

File AndroidManifest.xml mô tả tất cả các thành phần của ứng dụng. Các thành phần chẳng hạn như Activity, phải được khai báo trong file XML này. Trong các bài học khác, bạn sẽ chỉnh sửa file này để thêm các tính năng và quyền truy cập tính năng. Để tìm hiểu thêm, hãy xem [Tổng quan về App Manifest](#).

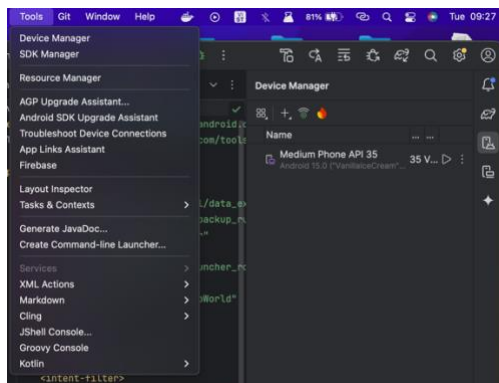
Task 3: Sử dụng thiết bị ảo (emulator)

Trong bài tập này, bạn sẽ sử dụng [Android Virtual Device \(AVD\) Manager](#) để tạo một thiết bị ảo (còn gọi là emulator) để mô phỏng cấu hình của một thiết bị Android và sử dụng thiết bị ảo này để chạy ứng dụng. Lưu ý rằng Android Emulator có các [yêu cầu bổ sung](#) ngoài các yêu cầu hệ thống cơ bản của Android Studio.

Sử dụng AVD Manager, bạn có thể định nghĩa các đặc điểm phần cứng của thiết bị, API level, bộ nhớ, giao diện và các thuộc tính khác, sau đó lưu nó lại thành một thiết bị ảo. Với các thiết bị ảo, bạn có thể test ứng dụng trên các cấu hình thiết bị khác nhau (ví dụ như máy tính bảng và điện thoại) với các API level khác nhau mà không cần phải sử dụng thiết bị thật.

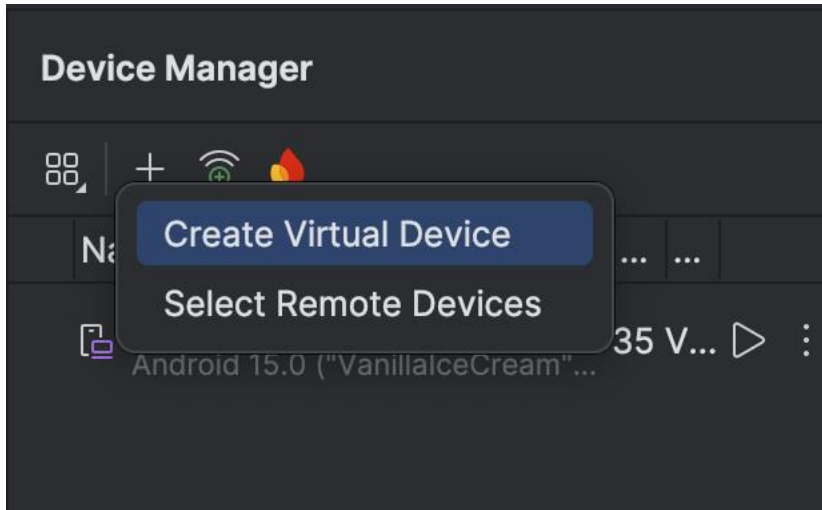
3.1 Thiết bị ảo có sẵn khi tạo dự án

Chọn tool > Device Manager trên thanh tool hoặc biểu tượng Device Manager ở thanh side bar bên phải, bạn có thể thấy 1 máy ảo mặc định được tạo khi khởi tạo dự án.



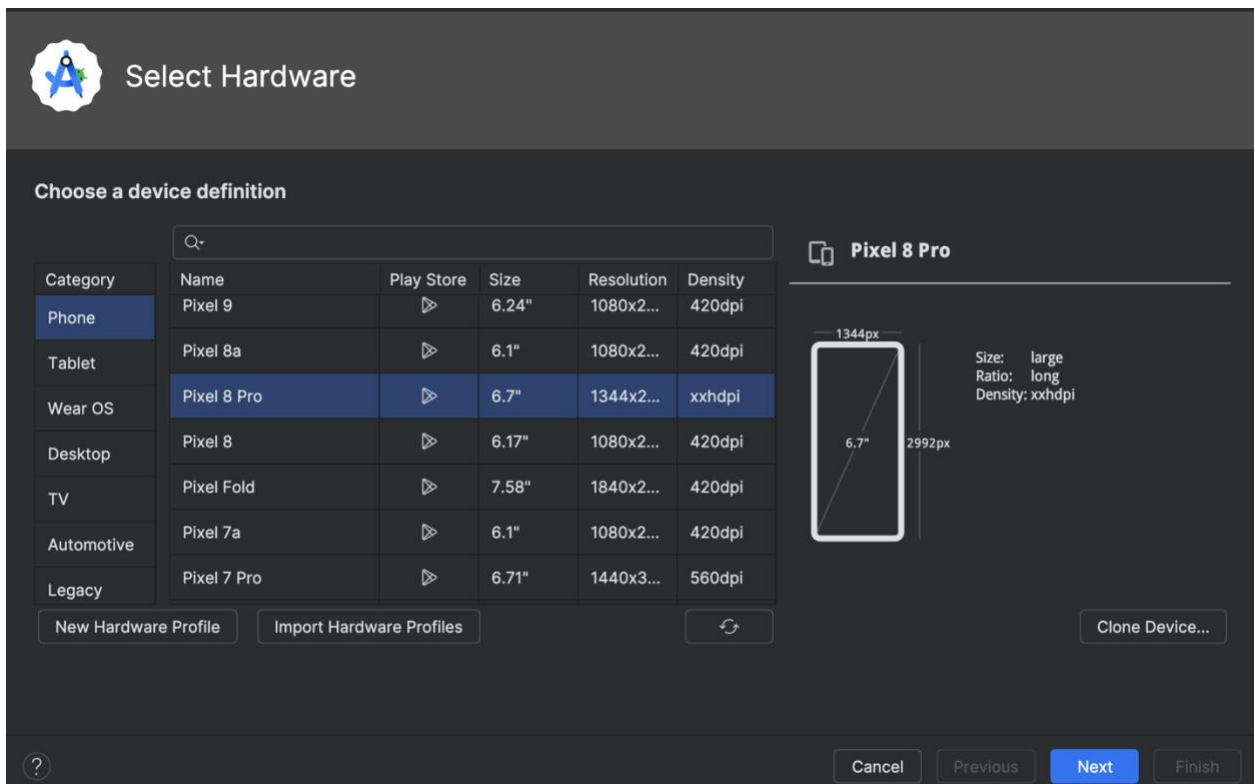
3.2 Tạo một thiết bị ảo Android (AVD)

1. Ấn vào dấu  và chọn **Create Virtual Device**

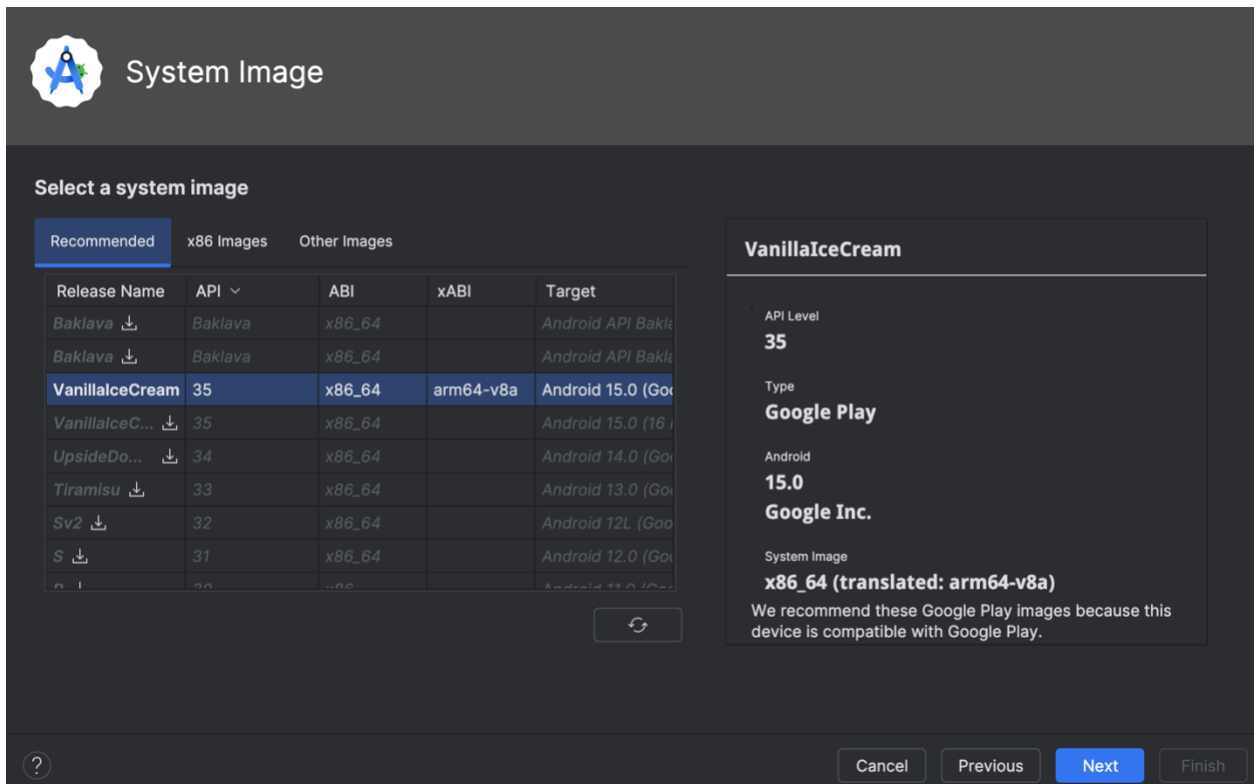


2. Ở cửa sổ **Virtual Device Configuration**, có thể chọn 1 máy ảo có sẵn hoặc tạo mới

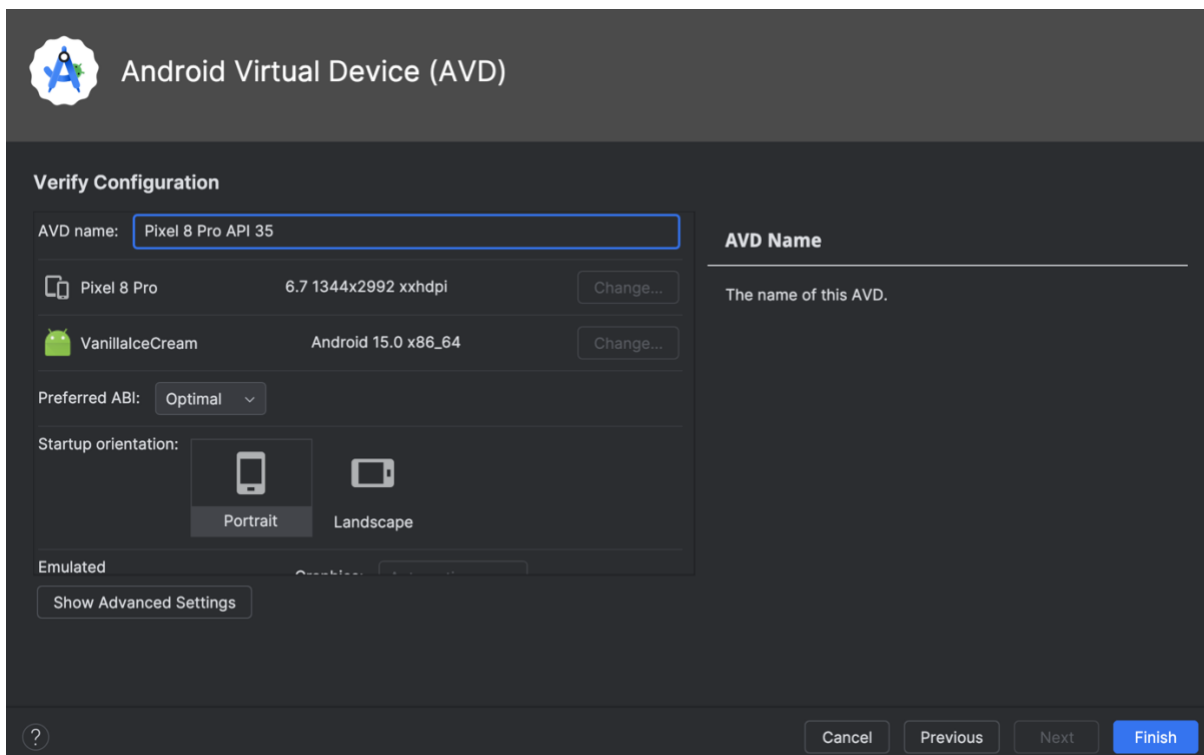
3.2.1 Có thể chọn 1 máy ảo có sẵn với thông số mà bạn muốn rồi ấn **Next**



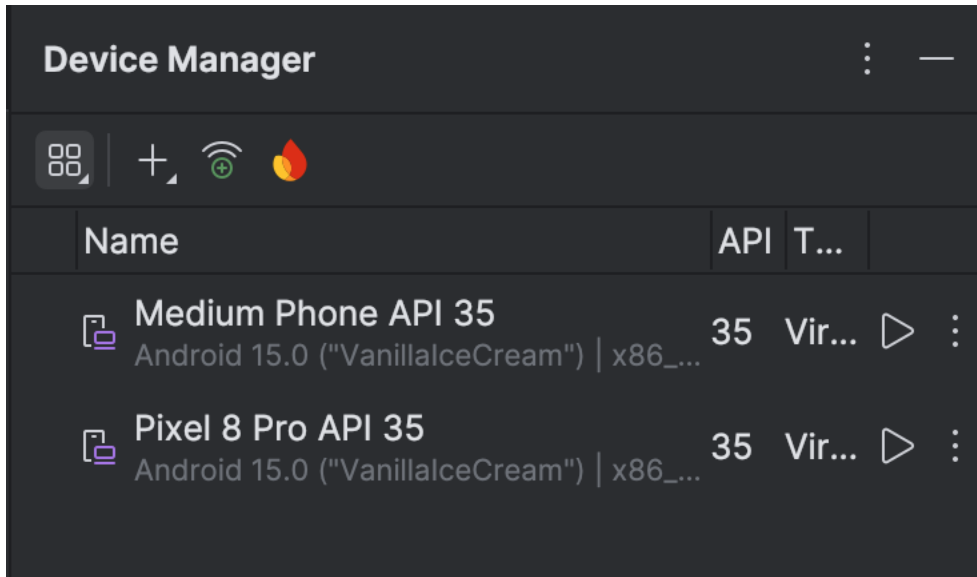
3.2.2 Tiếp tục **next** nếu không có lựa chọn khác



3.2.3 Nếu không có lựa chọn nào muốn thay đổi, tiếp tục ấn **finish**

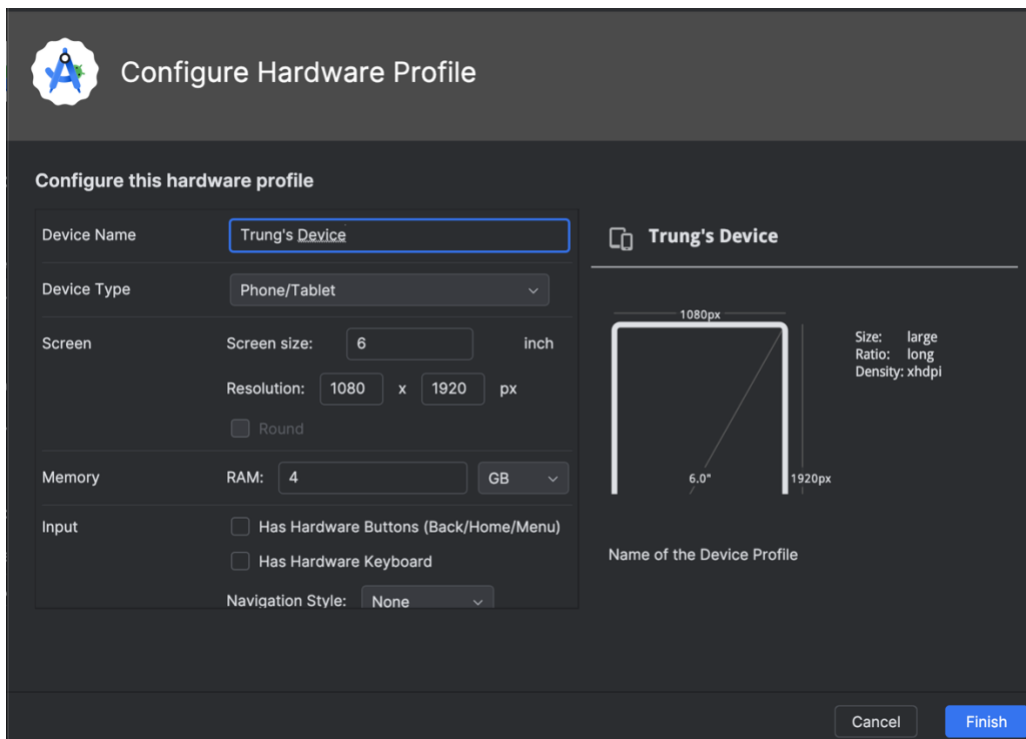


3.2.4 Màn hình **Device Manager** thêm thành công thiết bị vừa thêm



2.2.1 Nếu muốn tự cấu hình mới 1 thiết bị ảo, click **New Hardware Profile**. Nhập những thông tin bạn cần setup cho máy ảo của bạn.

Note: từ 7 inch là tablet còn dưới 7 inch là phone



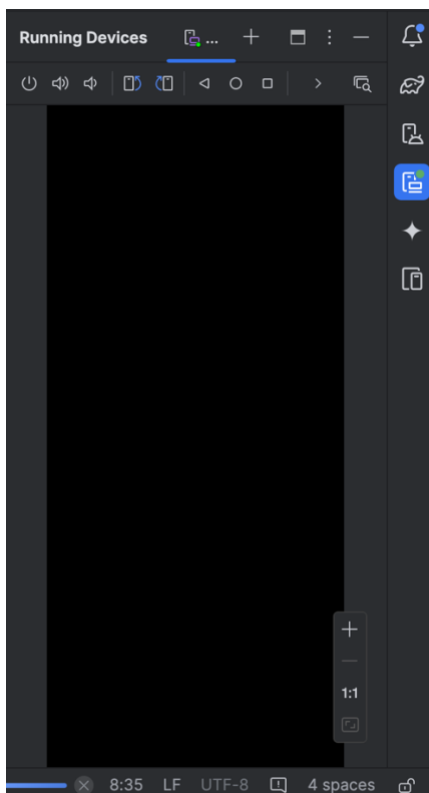
2.2.2 Sau khi ấn **Finish**, thiết bị được thêm vào danh sách thiết bị ảo

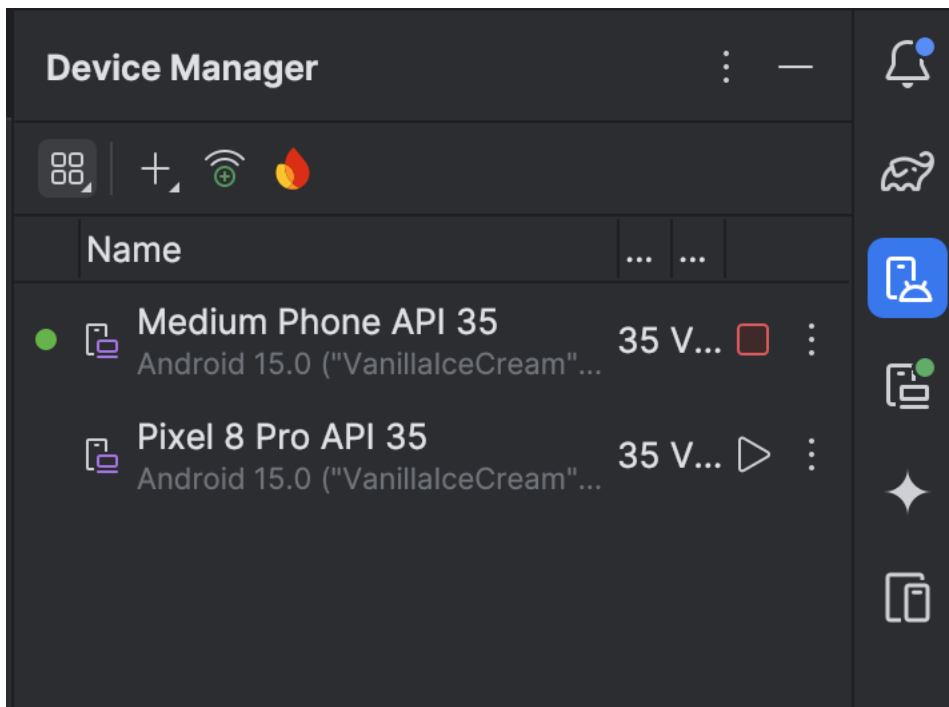
Choose a device definition					
Category	Name	Play Store	Size	Resolution	Density
Phone	Trung's Device		6.0"	1080x1...	xhdpi
Tablet	Pixel 9 Pro XL		6.8"	1344x2...	xxhdpi
Wear OS	Pixel 9 Pro Fold		8.0"	2076x2...	390dpi
Desktop	Pixel 9 Pro		6.3"	1280x2...	xxhdpi

3.3 Chạy chương trình với máy ảo

Trong task này chúng ta sẽ chạy chương trình Hello World !

1. Run > Run 'app' hoặc ấn vào biểu tượng  để chạy dự án.
2. Android studio sẽ chọn máy ảo đầu tiên trên list máy ảo trong Device Manager hoặc bạn có thể tự chọn 1 máy ảo.



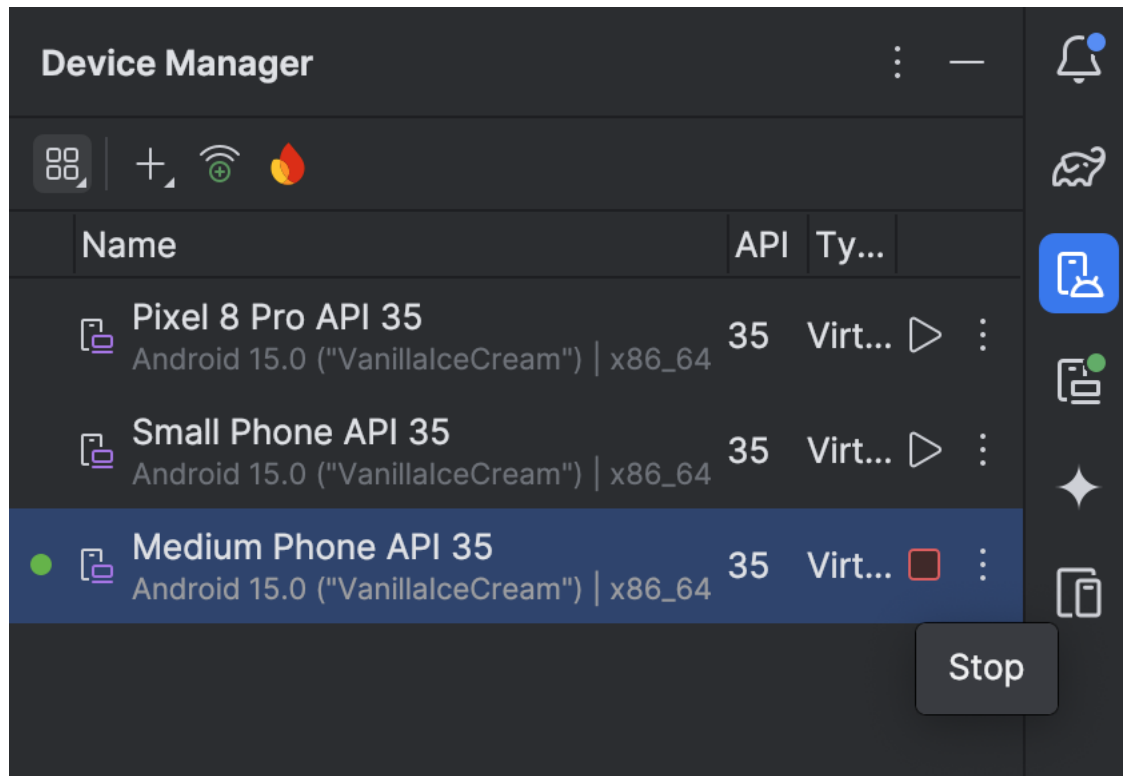


Trình giả lập sẽ khởi động giống như thiết bị thật. Tùy vào tốc độ máy tính của bạn, quá trình này có thể mất một lúc. Ứng dụng sẽ được build và khi trình giả lập sẵn sàng Android Studio sẽ tải ứng dụng lên trình giả lập và chạy nó.

Bạn sẽ thấy ứng dụng Hello World như hình dưới đây.



Tip: Khi test trên thiết bị ảo, mỗi khi làm việc nên khởi động ngay lúc đầu. Đừng tắt nó đi cho đến khi bạn test xong ứng dụng để không phải khởi động lại từ đầu. Để tắt thiết bị ảo, bạn click vào nút **Power**  hoặc nhấn **Stop** thiết bị trong **Device Manager**.



Task 4: (Tuỳ chọn) Sử dụng máy thật (không có nên dịch y như doc cũ)

Trong nhiệm vụ này, bạn sẽ chạy ứng dụng trên một thiết bị di động vật lý, chẳng hạn như điện thoại hoặc máy tính bảng. Bạn nên luôn kiểm tra ứng dụng trên cả thiết bị ảo và thiết bị vật lý.

Những gì bạn cần:

- Một thiết bị Android, chẳng hạn như điện thoại hoặc máy tính bảng.
- Cáp dữ liệu để kết nối thiết bị Android với máy tính.
- Nếu bạn đang sử dụng hệ điều hành **Linux** hoặc **Windows**, có thể cần thực hiện thêm một số bước để chạy ứng dụng trên thiết bị phần cứng. Đọc tài liệu [Using Hardware Devices](#). Bạn có thể sẽ cần cài đặt USB driver phù hợp cho thiết bị. Với trình điều khiển USB trên Windows, xem hướng dẫn tại [OEM USB Drivers](#).

4.1 Bật USB debugging

Để Android Studio giao tiếp với thiết bị của bạn, bạn cần bật USB Debugging trên thiết bị Android. Tùy chọn này có trong phần Developer options.

Trên **Android 4.2 trở lên**, **Developer options** mặc định bị ẩn. Để hiển thị tùy chọn nhà phát triển và bật **USB Debugging**, hãy làm theo các bước sau:

- Mở **Cài đặt (Settings)** => tìm **Giới thiệu về điện thoại (About phone)** => nhấn **Số bản dựng (Build number)** bảy lần liên tiếp.
- **Tùy chọn nhà phát triển (Developer options)** sẽ xuất hiện trong danh sách. Nhấn vào **Developer options**.
- Tìm và bật **Gỡ lỗi USB (USB Debugging)**

4.2 Chạy ứng dụng trên thiết bị

Bây giờ bạn có thể kết nối thiết bị của mình và chạy ứng dụng từ Android Studio.

1. Kết nối thiết bị với máy tính phát triển bằng cáp USB.
2. Nhấp vào nút **Run** trên thanh công cụ.
3. Chọn thiết bị kết nối. Ấn Ok.

Troubleshooting

Nếu Android Studio không nhận diện được thiết bị của bạn, hãy thử các cách sau:

1. Rút cáp USB và cắm lại.
2. Khởi động lại Android Studio.

Nếu máy tính vẫn không nhận diện được thiết bị hoặc hiển thị trạng thái "unauthorized", làm theo các bước sau:

1. Rút cáp USB khỏi thiết bị.
2. Trên thiết bị, mở Developer Options trong ứng dụng Cài đặt (Settings).
3. Nhấn vào Revoke USB Debugging authorizations (Thu hồi quyền gỡ lỗi USB).
4. Kết nối lại thiết bị với máy tính.
5. Khi hộp thoại yêu cầu cấp quyền xuất hiện, nhấn Cho phép (Allow).

Bạn có thể cần cài đặt **trình điều khiển USB (USB driver)** phù hợp cho thiết bị của mình. Xem tài liệu [Using Hardware Devices](#).

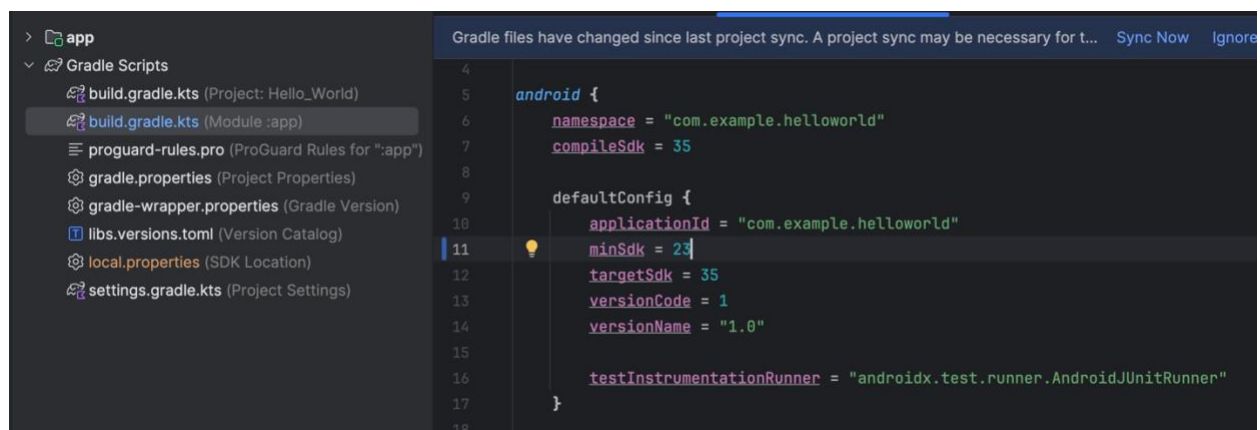
Task 5: Thay đổi cấu hình Gradle của ứng dụng

Trong task này, bạn sẽ chỉnh sửa một số cài đặt trong tệp **build.gradle.kts (Module: app)** để hiểu cách thay đổi cấu hình ứng dụng và đồng bộ hóa với Android Studio.

5.1 Thay đổi phiên bản SDK thấp nhất cho ứng dụng

Làm theo các bước sau:

1. Mở thư mục Gradle Scripts nếu chưa mở, sau đó mở tệp build.gradle (Module: app). Nội dung của tệp sẽ xuất hiện trong trình chỉnh sửa code.
2. Trong khối defaultConfig, thay đổi giá trị của minSdk thành 23, như ví dụ dưới đây (ban đầu được đặt là 24).



Tên thanh thông báo hiện ra sync now, ấn vào

5.2 Đồng bộ hóa cấu hình Gradle mới

Khi thay đổi cấu hình build, Android Studio yêu cầu đồng bộ các tệp dự án để có thể nhập các thay đổi cấu hình build và kiểm tra để đảm bảo cấu hình không gây ra lỗi biên dịch.

Để đồng bộ hóa tệp dự án, ấn vào Sync Now trên thanh thông báo xuất hiện sau khi thay đổi (như hình minh họa ngay trên) hoặc ấn vào biểu tượng Sync Project with

Gradle Files  trên thanh công cụ.

Khi quá trình đồng bộ Gradle hoàn tất, thông báo "Gradle build finished" sẽ xuất hiện ở góc dưới bên trái cửa sổ Android Studio.

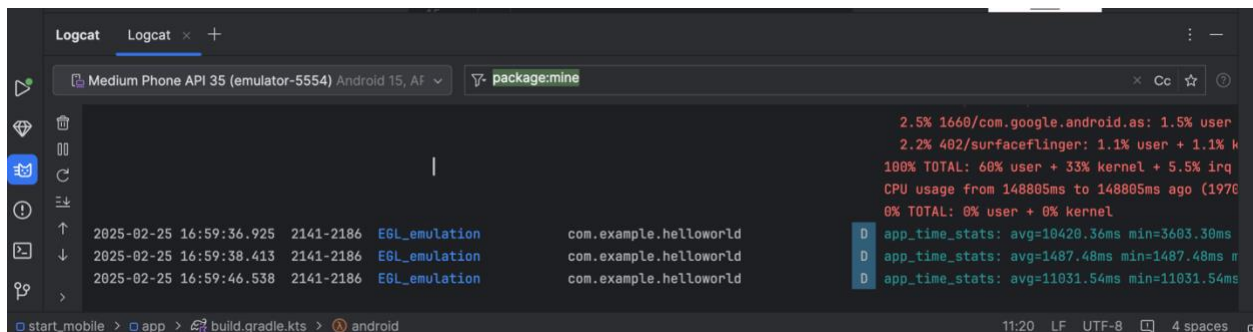
Để tìm hiểu sâu hơn về Gradle, hãy xem tài liệu [Build System Overview](#) và [Configuring Gradle Builds](#).

Task 6: Thêm câu lệnh log vào ứng dụng của bạn

Trong nhiệm vụ này, bạn sẽ thêm Log statements vào ứng dụng để hiển thị thông báo trong Logcat pane. Log messages là một công cụ mạnh mẽ giúp gỡ lỗi (debug), kiểm tra giá trị biến, luồng thực thi, và báo cáo ngoại lệ.

6.1 Xem Logcat pane

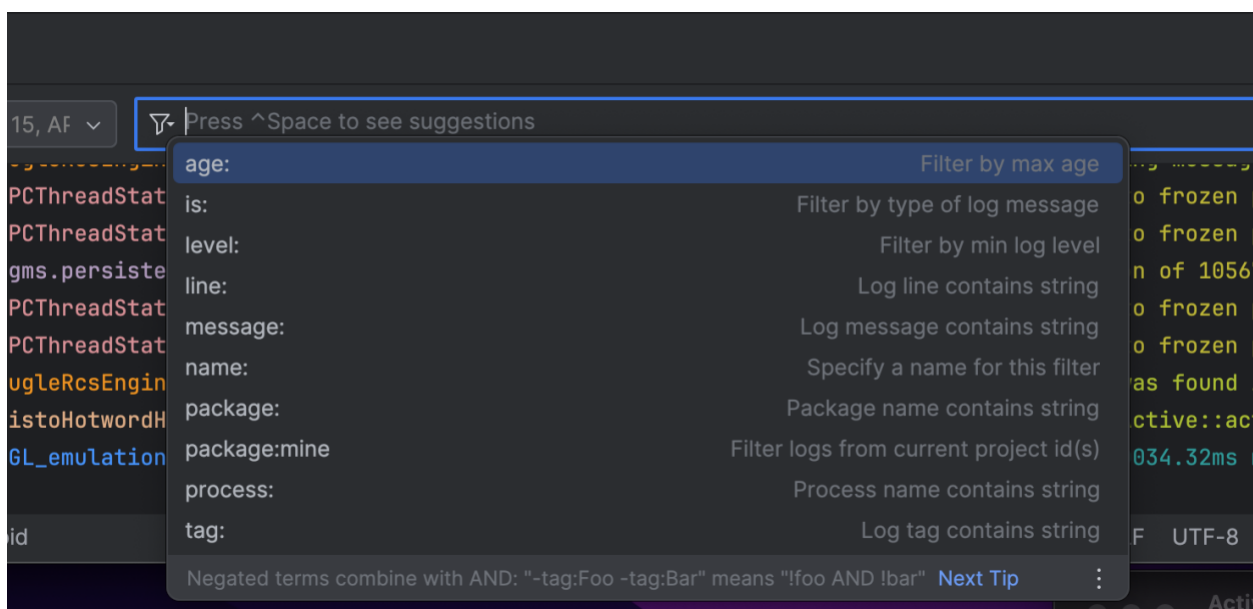
Để xem Logcat pane, hãy nhấp vào tab Logcat ở phía dưới cửa sổ Android Studio, như trong hình minh họa bên dưới.



Trong hình minh họa bên trên:

- **Logcat** dùng để mở và đóng **Logcat pane**, nơi hiển thị thông tin về ứng dụng khi đang chạy. Nếu bạn thêm các câu lệnh **Log** vào ứng dụng, các thông báo **Log messages** sẽ xuất hiện ở đây.

- Ở filter, ấn tổ hợp phím ctrl + space để xem gợi ý, có các option chọn log level ...



6.2 Thêm câu lệnh log vào ứng dụng của bạn

Các câu lệnh **log** trong mã nguồn ứng dụng sẽ hiển thị thông báo trong **Logcat pane**. Ví dụ:

```
Log.d( tag: "MainActivity", msg: "Hello World");
```

Các phần của một thông báo log:

- Log: lớp gửi thông điệp đến Logcat
- d: log ở mức debug, lọc các log mức debug trên logcat
- “MainActivity”: Đối số đầu tiên là **TAG**, giúp lọc thông báo trong **Logcat pane**. Thông thường, TAG là tên của Activity chứa thông báo log, nhưng bạn có thể đặt bất kỳ giá trị nào tiện gỡ lỗi.

Theo quy ước, **TAG** thường được định nghĩa là một hằng số trong **Activity**:

```
private static final String LOG_TAG = MainActivity.class.getSimpleName();
```

- **"Hello World"**: Đối số thứ hai là **thông báo của chương trình**

Thực hiện các bước sau:

- Mở ứng dụng **Hello World** trong **Android Studio**, sau đó mở tệp MainActivity.java.
- Để tự động thêm các import cần thiết (chẳng hạn như android.util.Log để sử dụng Log), làm như sau:
 - + Windows: Chọn File > Settings.
 - + macOS: Chọn Android Studio > Preferences.
- hazz

Haizzz

1.2) Giao diện người dùng tương tác đầu tiên

Giới thiệu

Giao diện người dùng (UI) xuất hiện trên màn hình của một thiết bị Android bao gồm một hệ thống phân cấp các đối tượng gọi là views — mọi thành phần trên màn hình đều là một View. Lớp View đại diện cho khối xây dựng cơ bản của tất

cả các thành phần giao diện người dùng và là lớp cơ sở cho các lớp cung cấp các thành phần UI tương tác như nút, checkboxes, và trường nhập văn bản.

Các lớp con **View** được sử dụng phổ biến sẽ được mô tả trong bài học:

- TextView để hiển thị văn bản.
- EditText để cho phép người dùng nhập và chỉnh sửa văn bản.
- Button và các phần tử có thể nhấp khác (như RadioButton, CheckBox, và Spinner) để cung cấp hành vi tương tác.
- ScrollView và RecyclerView để hiển thị các mục có thể cuộn.
- ImageView để hiển thị hình ảnh.
- ConstraintLayout và LinearLayout để chứa các phần tử View khác và định vị chúng.

Code Java hiển thị và điều khiển giao diện người dùng (UI) được chứa trong một lớp mở rộng từ Activity. Một Activity thường được liên kết với một bố cục giao diện người dùng được định nghĩa dưới dạng tệp XML (eXtended Markup Language). File XML này thường được đặt tên theo Activity định nghĩa bố cục của View trên màn hình.

Ví dụ, code MainActivity trong ứng dụng Hello World hiển thị một bố cục được định nghĩa trong file bố cục activity_main.xml, trong đó có một TextView với nội dung "Hello World".

Trong các ứng dụng phức tạp hơn, một Activity có thể thực hiện các hành động để phản hồi thao tác nhấn của người dùng, vẽ nội dung đồ họa hoặc yêu cầu dữ liệu từ cơ sở dữ liệu hoặc internet. Bạn sẽ tìm hiểu thêm về lớp Activity trong một bài học khác.

Trong bài thực hành này, bạn sẽ học cách tạo ứng dụng tương tác đầu tiên của mình - một ứng dụng cho phép người dùng tương tác. Bạn sẽ tạo một ứng dụng bằng cách sử dụng mẫu Empty Activity. Bạn cũng sẽ học cách sử dụng trình chỉnh sửa bố cục (layout editor) để thiết kế bố cục, cũng như cách chỉnh sửa bố cục trong XML. Bạn cần phát triển những kỹ năng này để có thể hoàn thành các bài thực hành khác trong khóa học này.

Những gì bạn nên biết trước:

Bạn nên thạo:

- Cách cài đặt và mở Android Studio.
- Cách tạo ứng dụng HelloWorld.
- Cách chạy ứng dụng HelloWorld.

Bạn sẽ học được:

- Cách tạo một ứng dụng có hành vi tương tác.
- Cách sử dụng trình chỉnh sửa bố cục (layout editor) để thiết kế giao diện.
- Cách chỉnh sửa bố cục trong XML.
- Nhiều thuật ngữ mới.

Những gì bạn sẽ làm:

- Tạo một ứng dụng và thêm hai Button cùng một TextView vào bố cục.
- Điều chỉnh từng phần tử trong ConstraintLayout để ràng buộc chúng với lề và các phần tử khác.
- Thay đổi thuộc tính của các phần tử giao diện người dùng (UI).
- Chỉnh sửa bố cục của ứng dụng trong XML.
- Trích xuất chuỗi hardcoded thành chuỗi tài nguyên.
- Triển khai phương thức xử lý sự kiện khi nhấn nút (click-handler methods) để hiển thị thông báo trên màn hình khi người dùng nhấn vào mỗi Button.

Tổng quan:

Ứng dụng **HelloToast** bao gồm hai phần tử **Button** và một **TextView**. Khi người dùng nhấn vào nút đầu tiên, ứng dụng sẽ hiển thị một thông báo ngắn (**Toast**) trên màn hình. Khi nhấn vào nút thứ hai, bộ đếm số lần nhấn sẽ tăng lên và hiển thị trong **TextView**, bắt đầu từ số 0.

Dưới đây là giao diện của ứng dụng sau khi hoàn thành:

Haizz

Nhiệm vụ 1: Tạo và khám phá một dự án mới

Trong bài thực hành này, bạn sẽ thiết kế và triển khai một dự án cho ứng dụng HelloToast. Một liên kết đến mã nguồn giải pháp sẽ được cung cấp sau.

1.1 Tạo dự án Android Studio

Khởi động Android Studio và tạo một dự án mới với các tham số sau:

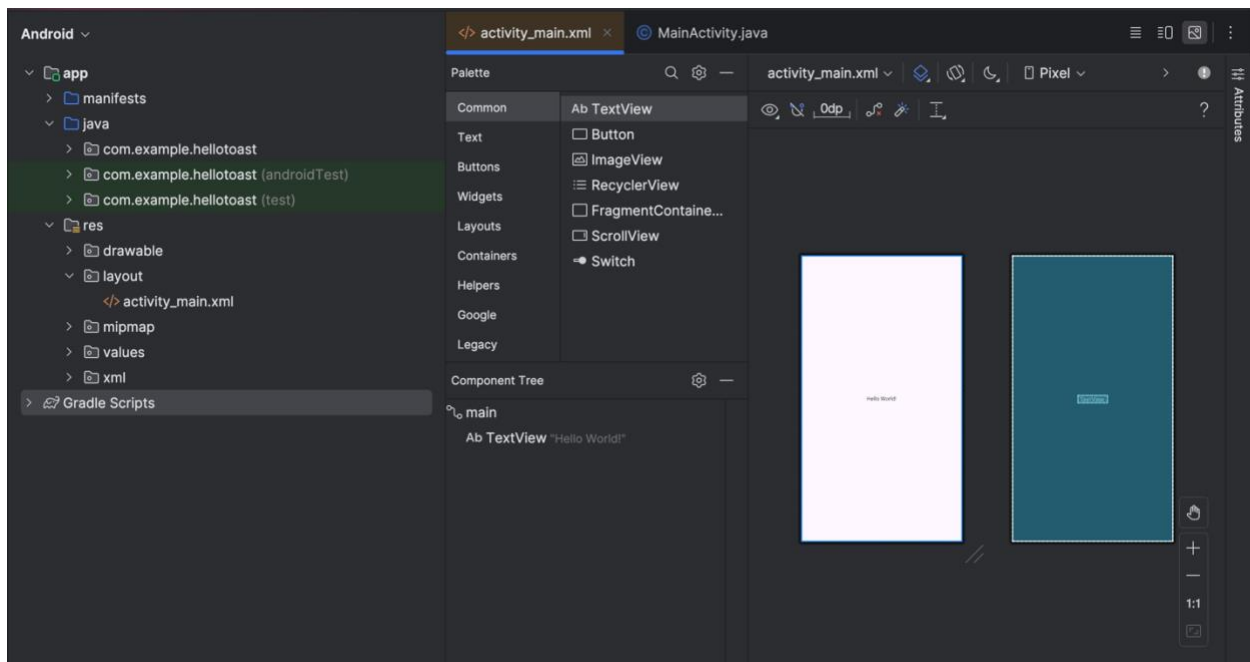
Thuộc tính	Giá trị
Application Name	Hello Toast
Phone and Tablet Minimum SDK	25
Template	Empty Activity
Generate Layout file box	Chọn
Backwards Compatibility box	Chọn
Company Name	com.example.android (hoặc tên miền của bạn)

Chọn **Run > Run app** hoặc nhấp vào biểu tượng **Run** trên thanh công cụ để biên dịch và chạy ứng dụng trên trình giả lập hoặc thiết bị của bạn.

1.2 Khám phá trình chỉnh sửa bố cục (Layout Editor)

Android Studio cung cấp trình chỉnh sửa bố cục để nhanh chóng xây dựng bố cục giao diện người dùng (UI) của ứng dụng. Nó cho phép bạn kéo thả các phần tử vào chế độ xem thiết kế trực quan và bản vẽ bố cục, định vị chúng trong bố cục, thêm ràng buộc và đặt thuộc tính. Ràng buộc xác định vị trí của một phần tử UI trong bố cục. Một ràng buộc đại diện cho một kết nối hoặc sự căn chỉnh với một phần tử khác, với bố cục cha, hoặc với một đường hướng dẫn vô hình.

Khám phá trình chỉnh sửa bố cục và tham khảo hình minh họa bên dưới khi bạn làm theo các bước được đánh số:



1. Trong thư mục **app > res > layout** ở bảng **Project > Android**, ấn vào tệp **activity_main.xml** để mở, nếu nó chưa được mở.
2. Ấn vào tab **Design** nếu nó chưa được chọn. Bạn sử dụng tab **Design** để thao tác với các phần tử và bố cục, còn tab **Text** để chỉnh sửa mã XML của bố cục.
3. **Pane Palettes** hiển thị các phần tử giao diện người dùng (UI) mà bạn có thể sử dụng trong bố cục của ứng dụng.
4. **Pane Component Tree** hiển thị cấu trúc phân cấp của các phần tử UI. Các phần tử giao diện người dùng được tổ chức theo cây phân cấp giữa cha và con, trong đó phần tử con kế thừa các thuộc tính của phần tử cha. Trong hình minh họa, **TextView** là phần tử con của **ConstraintLayout**. Bạn sẽ tìm hiểu thêm về các phần tử này trong bài học sau.
5. **Pane thiết kế và bản vẽ bố cục** của trình chỉnh sửa bố cục hiển thị các phần tử UI trong bố cục. Trong hình minh họa, bố cục chỉ hiển thị một phần tử: một **TextView** hiển thị dòng chữ "Hello World".
6. **Tab Attributes** hiển thị **Pane Attributes**, nơi bạn có thể thiết lập các thuộc tính cho một phần tử UI.

Mẹo: Xem **Building a UI with Layout Editor** để biết chi tiết về cách sử dụng trình chỉnh sửa bố cục, và **Meet Android Studio** để xem tài liệu đầy đủ về Android Studio.

Nhiệm vụ 2: Thêm các phần tử View trong trình chỉnh sửa bố cục

Trong nhiệm vụ này, bạn sẽ tạo bố cục giao diện người dùng (UI) cho ứng dụng HelloToast trong trình chỉnh sửa bố cục bằng cách sử dụng các tính năng của **ConstraintLayout**. Bạn có thể tạo các ràng buộc theo cách thủ công, như sẽ được hướng dẫn sau, hoặc tự động bằng công cụ **Autoconnect**.

2.1 Kiểm tra ràng buộc của phần tử

Làm theo các bước sau:

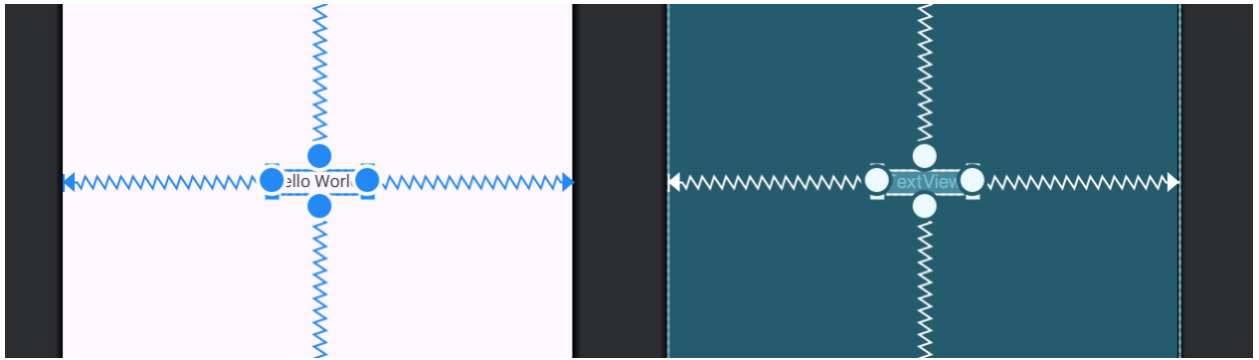
- Mở **activity_main.xml** trong **Project > Android** pane nếu nó chưa được mở. Nếu tab **Design** chưa được chọn, hãy ấn vào nó. Nếu không thấy **Blueprint**,

ấn vào nút **Select Design Surface**  trên thanh công cụ và chọn **Design and Blueprint**.

- Công cụ **Autoconnect** cũng nằm trên thanh công cụ và được bật theo mặc định. Ở bước này, hãy đảm bảo rằng công cụ này không bị tắt.
- Nhấp vào nút **Zoom in** để phóng to các bảng thiết kế và blueprint, giúp quan sát cận cảnh hơn.



- Chọn **TextView** trong bảng **Component Tree**. TextView có nội dung "Hello World" sẽ được tô sáng trong các bảng **design** và **blueprint**, đồng thời các ràng buộc của phần tử này cũng sẽ hiển thị.
- Thực hiện theo các bước. Ấn vào **chấm tròn** ở bên phải của TextView để **xóa ràng buộc ngang** đang liên kết phần tử với cạnh phải của bố cục. TextView sẽ **nhảy sang bên trái**, vì nó không còn bị ràng buộc với cạnh phải nữa. Để thêm lại ràng buộc ngang, nhấp vào cùng **chấm tròn** đó và **kéo một đường** đến cạnh phải của bố cục.



Trong bảng **blueprint** hoặc **design**, **handles** sau sẽ xuất hiện trên phần tử **TextView**:

Constraint handle: Để tạo một ràng buộc như trong hình minh họa ở trên, nhấp vào constrain handle (hiển thị dưới dạng **chấm tròn** ở cạnh phần tử). Sau đó, kéo tay cầm này đến một tay cầm ràng buộc khác hoặc đến **biên của phần tử cha**. Một đường **ziczac** sẽ hiển thị để đại diện cho ràng buộc.



Resizing handle: Để thay đổi kích thước phần tử, kéo **resizing handle** (hiển thị dưới dạng **hình vuông**). Khi kéo, tay cầm sẽ **chuyển thành góc xiên** để biểu thị rằng phần tử đang được điều chỉnh kích thước.

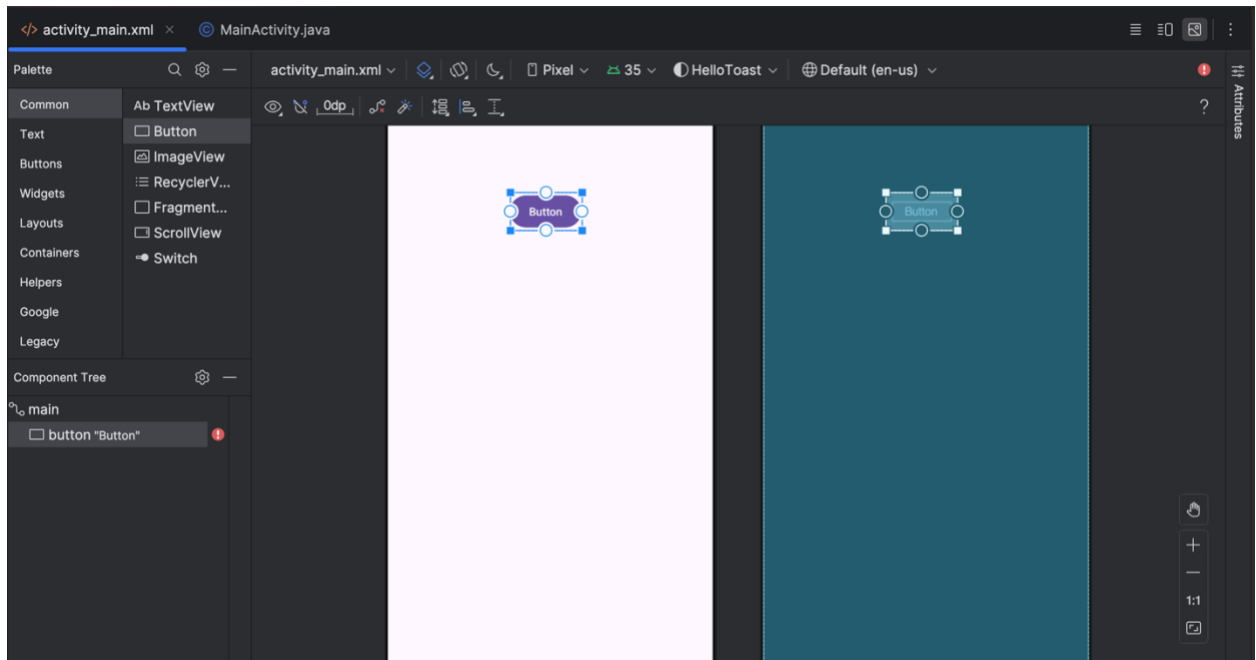


2.2 Thêm button vào bố cục

Khi được bật, công cụ **Autoconnect** tự động tạo hai hoặc nhiều ràng buộc cho một phần tử giao diện người dùng (**UI element**) với **parent layout**. Sau khi bạn kéo phần tử vào bố cục, nó sẽ tạo ràng buộc dựa trên vị trí của phần tử đó.

Thực hiện các bước sau để thêm một **Button**:

- Bắt đầu với một bố cục trống. Phần tử **TextView** không cần thiết, vì vậy khi nó vẫn đang được chọn, nhấn phím **Delete** hoặc chọn **Edit > Delete**.
- Kéo một Button vào bố cục. Kéo một **Button** từ **Palette pane** đến bất kỳ vị trí nào trong bố cục. Nếu bạn thả **Button** vào khu vực **trên cùng, giữa** của bố cục, có thể các ràng buộc (**constraints**) sẽ tự động xuất hiện. Nếu không, bạn có thể kéo các ràng buộc đến đỉnh, cạnh trái, và cạnh phải của bố cục.



2.3 Thêm một Button thứ hai vào bố cục

1. Kéo một Button khác từ bảng Palette vào giữa bố cục. **Autoconnect** có thể tự động tạo các ràng buộc ngang (**horizontal constraints**) cho bạn. Nếu không, bạn có thể tự kéo các ràng buộc ngang.
2. Kéo một ràng buộc dọc (vertical constraint) đến cạnh dưới của bố cục.

Bạn có thể **xóa ràng buộc** của một phần tử bằng cách chọn phần tử và ấn nút delete trên bàn phím hoặc muốn xóa tất cả ràng buộc ấn nút **Clear All**



Constraints

Task 3: Thay đổi thuộc tính của phần tử UI

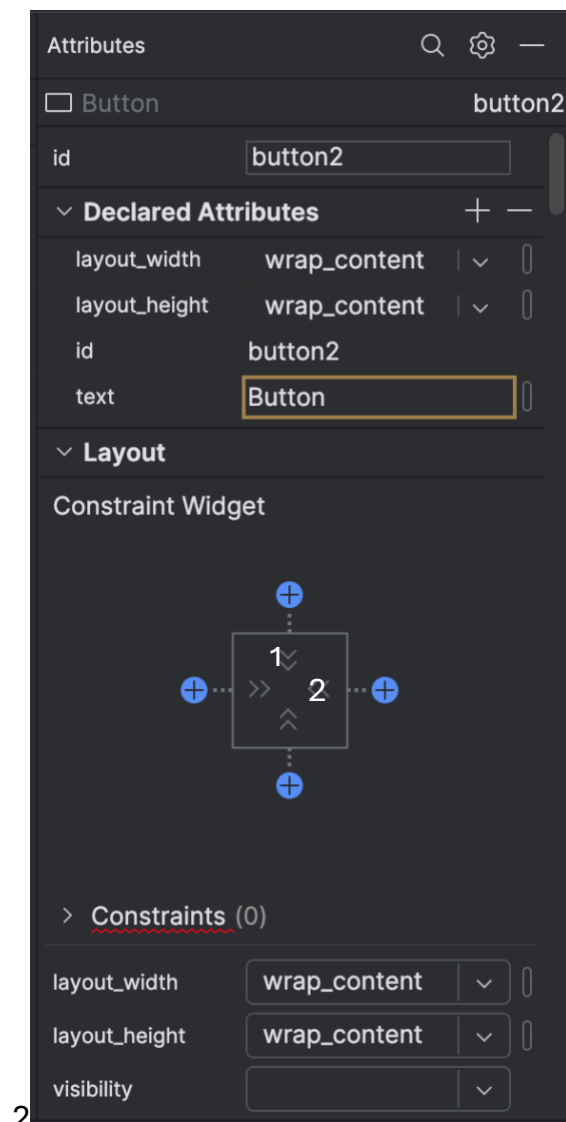
Trang thuộc tính cung cấp quyền truy cập vào tất cả các thuộc tính XML mà bạn có thể gán cho một phần tử UI. Bạn có thể tìm thấy các thuộc tính của tất cả các View trong View class documentation.

Trong nhiệm vụ này, bạn sẽ nhập các giá trị mới và thay đổi các giá trị cho các thuộc tính quan trọng của Button, những thuộc tính này cũng áp dụng cho hầu hết các loại View.

3.1 Change the Button size

Trình chỉnh sửa bố cục cung cấp các tay cầm thay đổi kích thước ở cả bốn góc của một View, giúp bạn có thể thay đổi kích thước View một cách nhanh chóng. Bạn có thể kéo các tay cầm ở mỗi góc của View để thay đổi kích thước, nhưng làm như vậy sẽ cố định kích thước chiều rộng và chiều cao. Tránh đặt kích thước cố định cho hầu hết các phần tử View, vì các kích thước cố định không thể thích ứng với nội dung và kích thước màn hình khác nhau.

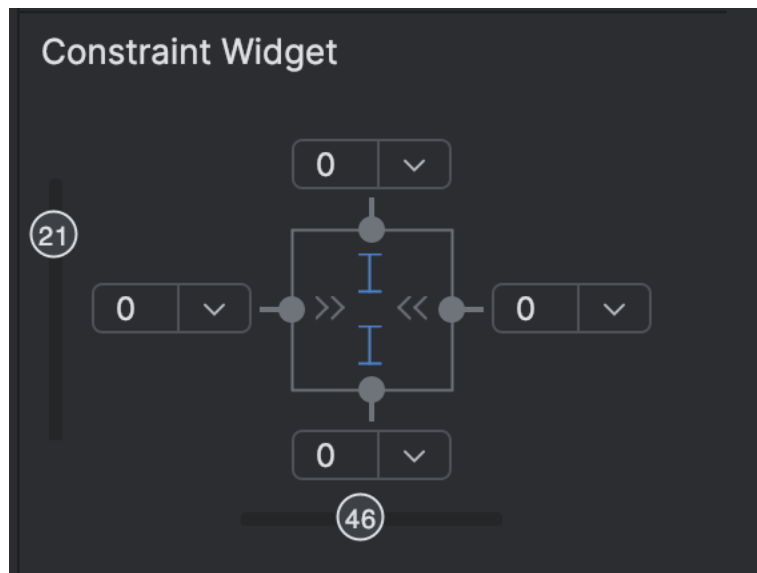
Thay vào đó, hãy sử dụng pane thuộc tính ở bên phải của trình chỉnh sửa bố cục để chọn một chế độ định kích thước không sử dụng các giá trị cố định. Pane Thuộc tính bao gồm một bảng điều khiển định kích thước hình vuông gọi là view inspector ở trên cùng. Các ký hiệu bên trong hình vuông biểu thị cài đặt chiều cao và chiều rộng như sau:



- (1) Height control: control này xác định thuộc tính `layout_height` và xuất hiện dưới dạng hai đoạn trên các cạnh trên và dưới của hình vuông. Các góc nghiêng chỉ ra rằng điều khiển này được đặt thành `wrap_content`, nghĩa là View sẽ mở rộng theo chiều dọc khi cần thiết để phù hợp với nội dung của nó.
- (2) Width control: control này xác định thuộc tính `layout_width` và xuất hiện dưới dạng hai đoạn trên các cạnh trái và phải của hình vuông. Các góc nghiêng chỉ ra rằng điều khiển này được đặt thành `wrap_content`, nghĩa là View sẽ mở rộng theo chiều ngang khi cần thiết để phù hợp với nội dung của nó
- Attribute pane close button: cái tên nói lên tất cả.

Làm theo các bước:

- Phía trên bên phải màn hình ấn attributes tab
- Ấn vào height control, lần đầu là đường thẳng để fixed, lần 2 là lò xo là match constraints



- Chọn Button thứ hai và thực hiện các thay đổi tương tự cho **layout_width** như ở bước trước.

Như đã thấy trong các bước trước, các thuộc tính **layout_width** và **layout_height** trong bảng **Attributes** sẽ thay đổi khi bạn điều chỉnh các điều khiển chiều rộng và chiều cao trong **inspector**. Những thuộc tính này có thể nhận một trong ba giá trị trong **ConstraintLayout**:

- **match_constraint:** Mở rộng phần tử View để lấp đầy phần tử cha theo chiều rộng hoặc chiều cao — đến giới hạn của **margin**, nếu có thiết lập. Phần tử cha trong trường hợp này là **ConstraintLayout**. Bạn sẽ tìm hiểu thêm về **ConstraintLayout** trong nhiệm vụ tiếp theo.
- **wrap_content:** Thu nhỏ kích thước của phần tử View sao cho nó vừa đủ chứa nội dung của nó. Nếu không có nội dung, phần tử View sẽ trở nên **vô hình**.
- **Fixed size:** Để chỉ định một kích thước cố định có thể điều chỉnh theo kích thước màn hình của thiết bị, hãy sử dụng đơn vị **density-independent pixels (dp)**. Ví dụ, **16dp** có nghĩa là **16 pixel độc lập với mật độ màn hình**.

Mẹo: Nếu bạn thay đổi thuộc tính **layout_width** bằng cách sử dụng menu popup của nó, giá trị của **layout_width** sẽ được đặt thành **0** vì không có kích thước cố định được thiết lập. Cài đặt này tương đương với **match_constraint**—phần tử có thể mở rộng tối đa để phù hợp với các ràng buộc (**constraints**) và thiết lập **margin**.

3.2 Thay đổi thuộc tính của Button

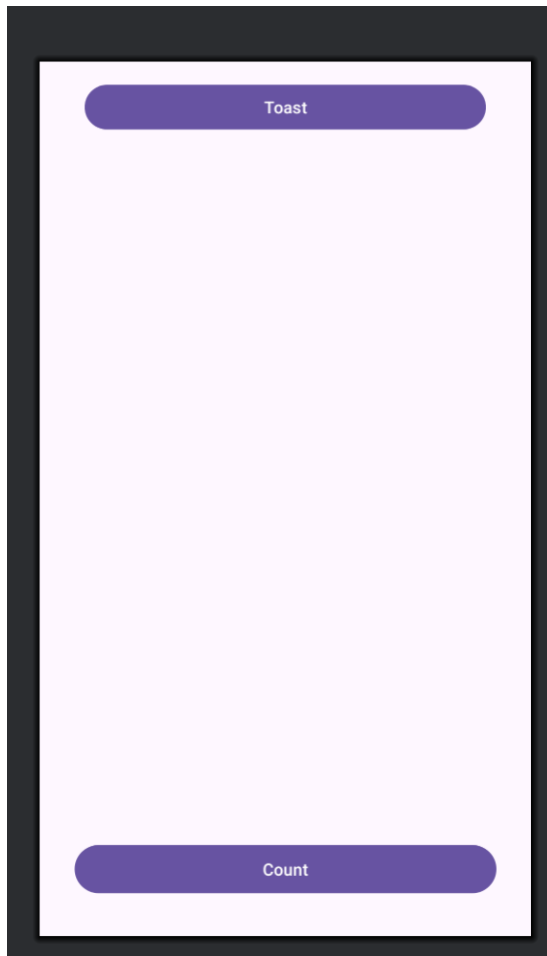
Để xác định duy nhất mỗi **View** trong một **Activity layout**, mỗi **View** hoặc **View subclass** (chẳng hạn như **Button**) cần có một **ID duy nhất**. Ngoài ra, để các **Button** có thể hoạt động, chúng cần có nội dung văn bản (**text**). Các phần tử **View** cũng có thể có **background** là màu sắc hoặc hình ảnh.

Pane Thuộc tính (Attributes) cung cấp quyền truy cập vào tất cả các thuộc tính có thể gán cho một **View**. Bạn có thể nhập giá trị cho từng thuộc tính như **android:id**, **background**, **textColor**, **text**.

Thực hiện theo các bước sau:

- Chọn **Button đầu tiên**, chỉnh sửa **ID** trong **Attributes pane** thành **button_toast** cho thuộc tính **android:id** (được sử dụng để xác định phần tử trong layout).
- Đặt thuộc tính **background** thành **@color/colorPrimary**. (Khi nhập **@c**, danh sách gợi ý sẽ xuất hiện để chọn nhanh).
- Đặt thuộc tính **textColor** thành **@android:color/white**.

- Chỉnh sửa thuộc tính **text** thành **Toast**.
- Thay đổi thuộc tính tương tự cho button thứ hai, id là `button_count`, text là `Count`, màu cho background và text như trên.



Màu `colorPrimary` là màu chính của chủ đề, một trong các màu cơ bản được định nghĩa sẵn trong tệp tài nguyên `colors.xml`. Nó thường được sử dụng cho thanh ứng dụng. Việc sử dụng các màu cơ bản này cho các thành phần giao diện khác giúp tạo ra một giao diện người dùng đồng nhất. Bạn sẽ tìm hiểu thêm về chủ đề ứng dụng (app themes) và Material Design trong bài học khác.

Task 4: Thêm một `TextEdit` và thiết lập các thuộc tính

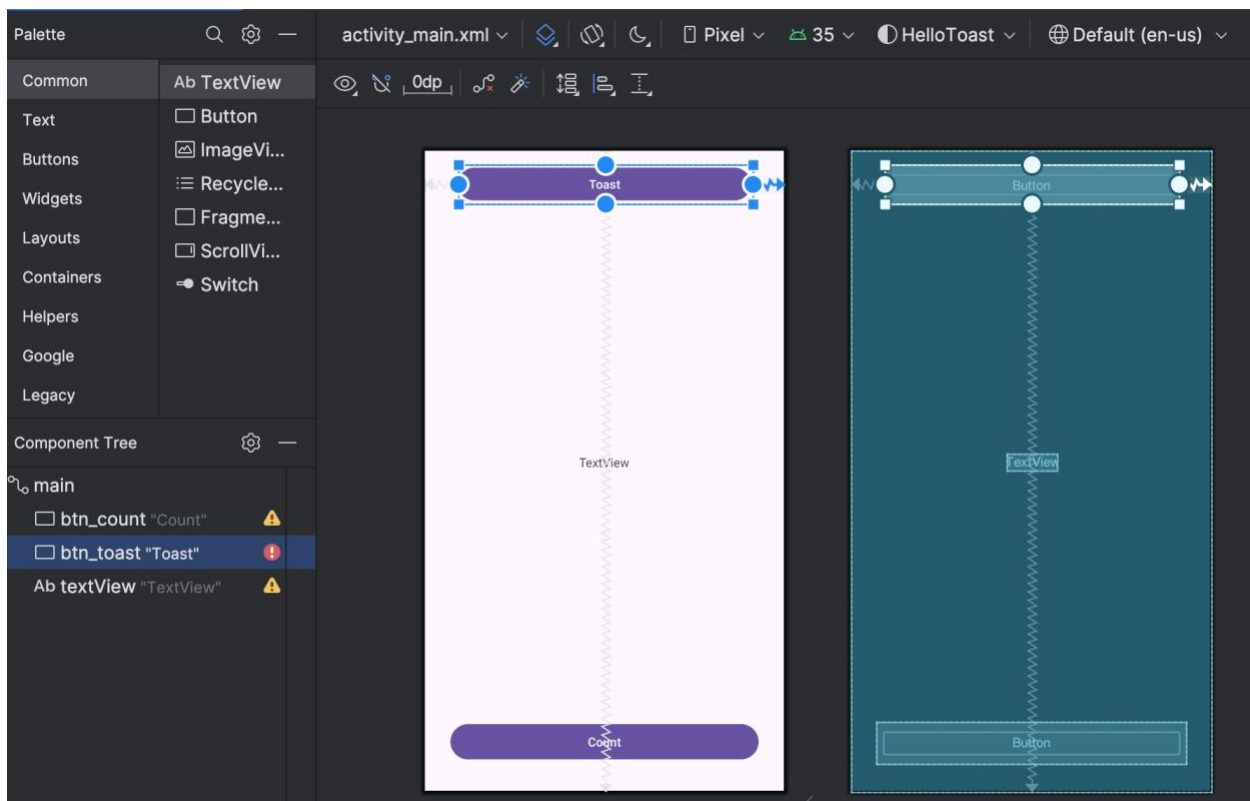
Một trong những lợi ích của `ConstraintLayout` là khả năng căn chỉnh hoặc ràng buộc các thành phần với các thành phần khác. Trong nhiệm vụ này, bạn sẽ thêm một `TextView` vào giữa bố cục và ràng buộc nó theo chiều ngang với các lề (margins) và

theo chiều dọc với hai Button. Sau đó, bạn sẽ thay đổi các thuộc tính của TextView trong bảng Attributes.

4.1 Thêm một TextView và thiết lập ràng buộc

1. Kéo một TextView từ bảng Palette vào phần trên của bố cục. Sau đó, kéo một ràng buộc từ cạnh trên của TextView đến cạnh dưới của nút Toast. Điều này sẽ làm cho TextView luôn nằm ngay bên dưới nút Toast.

2. kéo một ràng buộc từ cạnh dưới của TextView đến tay cầm ở cạnh trên của nút Count, và từ hai cạnh bên của TextView đến hai cạnh bên của bố cục. Điều này sẽ giữ TextView ở giữa bố cục, nằm giữa hai nút Toast và Count.

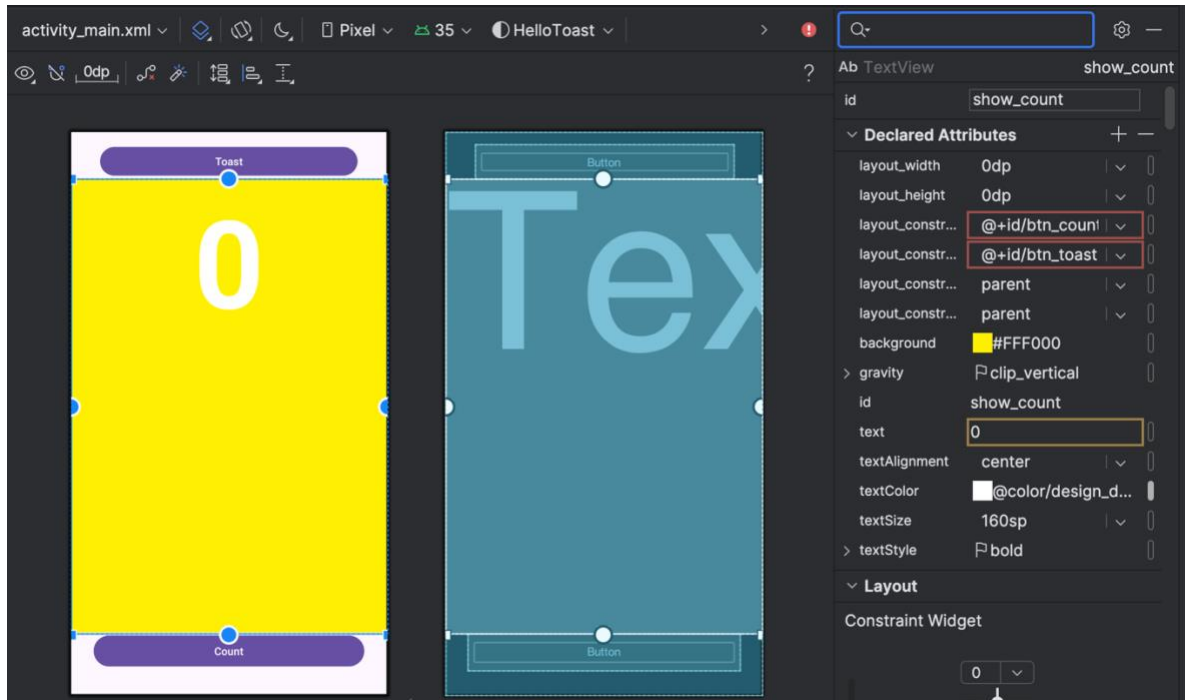


4.2 Đặt thuộc tính cho TextView

Với **TextView** được chọn, mở **Attributes** nếu các thuộc tính sau chưa được thiết lập hãy thiết lập như sau:

- Đặt **ID** thành show_count.
- Đặt **text** thành 0.
- Đặt **textSize** thành 160sp.

- Đặt **textStyle** thành **B (bold)** và **textAlignment** thành **CENTER** (căn giữa nội dung).
- Thay đổi **layout_width** và **layout_height** thành **match_constraint**.
- Đặt **textColor** thành **@color/design_default_color_primary**.
- Cuộn xuống bảng **Attributes**, nhấp vào **View all attributes**, sau đó cuộn tiếp đến thuộc tính **background** và nhập #FFF00 để đặt màu vàng.
- Cuộn xuống **gravity**, mở rộng tùy chọn **gravity**, và chọn **center_vertical**.

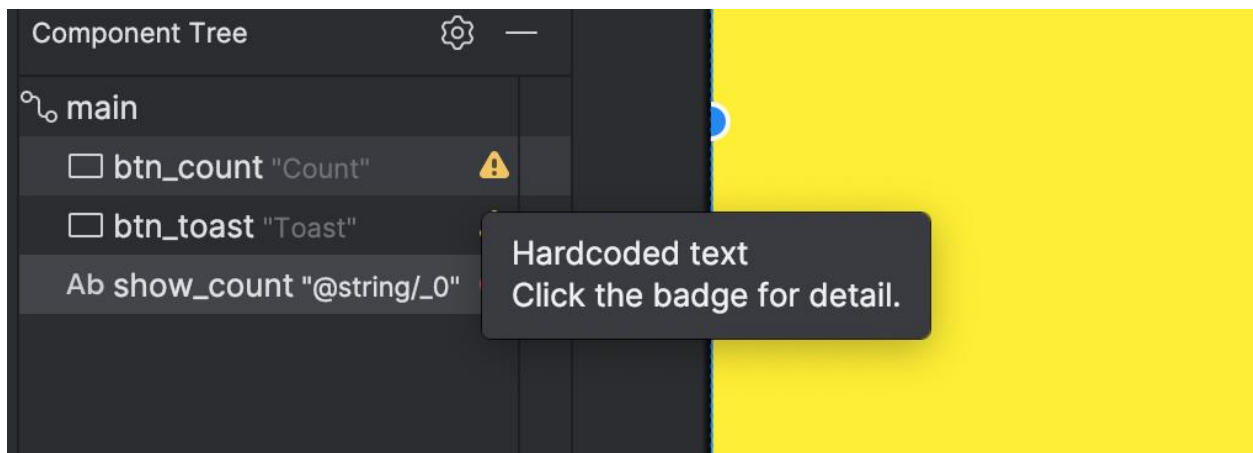


- **textSize**: Kích thước chữ của **TextView**. Trong bài học này, kích thước được đặt thành 160sp. **sp** (scale-independent pixels) là đơn vị tỷ lệ với mật độ màn hình và giống kích thước chữ của người dùng. Khi đặt kích thước phông chữ, nên sử dụng **sp** để đảm bảo chúng tự điều chỉnh phù hợp với cả mật độ màn hình và tùy chọn của người dùng.
- **textStyle** và **textAlignment**: textStyle trong bài này được đặt là B (bold) để in đậm, textAlignment được đặt thành **ALIGNCENTER** để căn giữa.
- gravity: xác định cách một **View** được căn chỉnh trong **ViewGroup** hoặc **View cha** của nó. Bài này **TextView** được căn giữa theo chiều dọc trong **ConstraintLayout** bằng cách đặt **gravity** thành **center_vertical**.

Lưu ý: Trong Attributes pane, một số thuộc tính phổ biến của Button sẽ xuất hiện ngay ở trang đầu, nhưng với TextView, các thuộc tính như background có thể nằm ở rất sâu phía dưới.

Task 5: Chỉnh sửa giao diện trong XML

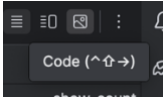
Bố cục ứng dụng Hello Toast gần hoàn thiện. Tuy nhiên, một dấu chấm than xuất hiện bên cạnh mỗi thành phần UI trong **Component Tree**. Di con trỏ qua các dấu chấm than này để xem các cảnh báo, như được hiển thị bên dưới. Cảnh báo với cả ba thành phần: “**Hardcoded text. Click the badge for detail.**”.



Cách dễ nhất để khắc phục các vấn đề bố cục là chỉnh sửa trực tiếp trong XML. Mặc dù **layout editor** là một công cụ mạnh mẽ, nhưng một số thay đổi sẽ dễ thực hiện hơn khi chỉnh sửa trực tiếp trong XML.

5.1 Mở mã XML cho bố cục

Trong bước này, hãy mở tệp **activity_main.xml** nếu nó chưa được mở, sau đó nhấp

vào tab code  ở cuối trình chỉnh sửa bố cục.

Trình chỉnh sửa XML sẽ xuất hiện, thay thế các bảng thiết kế (**design**) và sơ đồ (**blueprint**). Như bạn có thể thấy trong hình bên dưới, một phần mã XML của bố cục hiển thị các cảnh báo được đánh dấu.

Di chuột qua chuỗi mã hóa cứng "**Toast**" để xem thông báo cảnh báo.


```
<Button
    android:id="@+id/btn_count"
    android:layout_width="353dp"
    android:layout_height="48dp"
    android:text="Count"
    app:layout_constraintTop_toTopOf="parent"
    app:layout_constraintVertical_bias="0.887" />
```

Hardcoded string "Count", should use `@string` resource [Toggle info \(⌘F1\)](#)

[Extract string resource](#) [More actions...](#)

1.3) Trình chỉnh sửa bố cục

1.4) Văn bản và các chế độ cuộn

1.5) Tài nguyên có sẵn

Bài 2) Activities

2.1) Activity và Intent

2.2) Vòng đời của Activity và trạng thái

2.3) Intent ngầm định

Bài 3) Kiểm thử, gỡ lỗi và sử dụng thư viện hỗ trợ

3.1) Trình gỡ lỗi

3.2) Kiểm thử đơn vị

3.3) Thư viện hỗ trợ

CHƯƠNG 2. TRẢI NGHIỆM NGƯỜI DÙNG

Bài 1) Tương tác người dùng

- 1.1) Hình ảnh có thể chọn
- 1.2) Các điều khiển nhập liệu
- 1.3) Menu và bộ chọn
- 1.4) Điều hướng người dùng
- 1.5) RecyclerView

Bài 2) Trải nghiệm người dùng thú vị

- 2.1) Hình vẽ, định kiểu và chủ đề
- 2.2) Thẻ và màu sắc
- 2.3) Bố cục thích ứng

Bài 3) Kiểm thử giao diện người dùng

- 3.1) Espresso cho việc kiểm tra UI

CHƯƠNG 3. LÀM VIỆC TRONG NỀN

Bài 1) Các tác vụ nền

- 1.1) AsyncTask
- 1.2) AsyncTask và AsyncTaskLoader
- 1.3) Broadcast receivers

Bài 2) Kích hoạt, lập lịch và tối ưu hóa nhiệm vụ nền

- 2.1) Thông báo
- 2.2) Trình quản lý cảnh báo
- 2.3) JobScheduler

CHƯƠNG 4. LƯU DỮ LIỆU NGƯỜI DÙNG

Bài 1) Tùy chọn và cài đặt

1.1) Shared preferences

1.2) Cài đặt ứng dụng

Bài 2) Lưu trữ dữ liệu với Room

2.1) Room, LiveData và ViewModel

2.2) Room, LiveData và ViewModel